

PLANT LEAF DISEASE DETECTION SYSTEM

A PROJECT REPORT

Submitted by

Sadanand Kumar

*in partial fulfillment for the award of
the degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science Engineering



Centurion
UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

PARALAKHEMUNDI CAMPUS

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ODISHA

APRIL & 2025

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**DEPARTMENT OF COMPUTER SCIENCE
ENGINEERING SCHOOL OF ENGINEERING
AND TECHNOLOGY PARALAKHEMUNDI
CAMPUS**

BONAFIDE CERTIFICATE

Certified that this project report **PLANT LEAF DISEASE DETECTION SYSTEM** is the bonafide work of “**SADANAND KUMAR** ” who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

SIGNATURE

Prof. Debendra Maharana

Professor of Computer Science and Engineering

Certified that the above-mentioned project has been duly carried out as per the norms of the college and statutes of the university.

SIGNATURE

HEAD OF THE DEPARTMENT / DEAN OF THE SCHOOL

Professor of Computer Science and Engineering

DEPARTMENT SEAL

DECLARATION

I hereby declare that the project entitled “**PLANT LEAF DISEASE DETECTION SYSTEM**” submitted for the “Project” of 6th semester B.Tech in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student: Sadanand Kumar

Signature of the Student:

Registration No: 220101120034

Place: Paralakhemundi

Date:

ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to Prof. Debendra Maharana, Department of Computer Science and Engineering, SoET, Paralakhemundi Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank Dr. Dhawaleshwar Rao Ch, Head of the Dept. of Computer Science and Engineering , SoET, Paralakhemundi Campus and Dr. PRAFULLA KUMAR PANDA, Dean, School of Engineering and Technology, Paralakhemundi Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered during various stages of image interpretation by Prof. Debendra Maharana.

I am highly grateful to Prof. Debendra Maharana who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to Prof. Debendra Maharana for their constant encouragement, co-operation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavor.

Name of the Student: Sadanand Kumar

Signature of the Student:

Registration No: 220101120034

Place: Paralakhemundi

Date:

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGENO
	CERTIFICATE	i
	DECLARATION	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
1.	<i>CHAPTER – 1 Software Requirement Specification</i>	01-10
1.1.	Introduction	
1.1.1.	Project Description	
1.1.2.	Purpose	
1.1.3.	Scope	
1.1.4.	Overview	
1.1.5.	Key Features	
1.1.6.	Challenges	
1.1.7.	Glossary	
1.2.	Requirement Analysis	11-12
1.2.1.	Operating System	
1.2.2.	Deployment Tools	
1.2.3.	Backend Technologies	
1.2.4.	Frontend Technology	
1.2.5.	APIs and Protocols	
1.2.6.	Communication Protocols	
1.2.7.	Testing Tools	
1.2.8.	Main Blocks of PLANT LEAF DISEASE DETECTION SYSTEM	
1.2.9.	Block Diagram of PLANT LEAF DISEASE DETECTION SYSTEM Web Application	13
1.3.	Modules	14-15
1.3.1.	Admin Module	
1.3.2.	User Module	
1.4.	Use Case Diagram	16-17
1.4.1.	Admin	
1.4.2.	User	
2.	<i>CHAPTER – 2 Design and Analysis</i>	
2.1.	Entity Relationship Diagram	18
2.2.	Schema Diagram	19

2.3. Activity Flow Diagram	
2.3.1. Admin	20
2.3.2. User	21
2.3.3. Overview	22
2.4. Tables In Database	23

3. *CHAPTER 3 PROJECT WORK PART-I*

3.1. Front-End code	24-43
3.1.1.Landing Page HTML - navbar.component.html	
3.1.2.Landing Page HTMLM - navbar.component.CSS	
3.1.3.Landing Page HTML - navbar.component.ts	
3.1.4. home.component.html	
3.1.5. home.component.css	
3.1.6. home.component.ts	
3.1.7.register.component.html	
3.1.8.register.component.css	
3.1.9.register.component.ts	
3.1.10.login.component.html	
3.1.11.login.component.css	
3.1.12.login.component.ts	
3.1.13.userlist.component.html	
3.1.14.userlist.component.css	
3.1.15.userlist.component.ts	
3.2 Backend Code	44-58
3.2.1. SpringBootWithAngular17Application	
3.2.2. OurUsersMongo	
3.2.3. UsersMongoRepo	
3.2.4. UserManagementController	
3.2.5. UsersManagementService	
3.2.6. OurUserDetailsService	

- 3.2.7. JWTUtils
- 3.2.8. ReqRes
- 3.2.9. SecurityConfig
- 3.2.10. JWTAuthFilter
- 3.2.11. CorsConfig
- 3.2.12. application.properties

3.3 MACHINE LEARNING BACK-END 59-76

- 3.3.1 app.py
- 3.3.2 CNN.py
- 3.3.3 base.html
- 3.3.4 home.html
- 3.3.5 contact-us.html
- 3.3.6 market.html
- 3.3.7 index.html
- 3.3.8 submit.html

4. *CHAPTER – 4* *PROJECT WORK OUTPUTS* 77-82

4. CHAPTER – 5 SUMMARY & SCOPE FOR FURTHER STUDY

83-86

4.1. Summary

4.2. Scope for Further Study

4.2.1. Integration of Advanced AI/ML Models

4.2.2. IoT and Sensor Integration

4.2.3. Mobile and Offline Functionality

4.2.4. Cloud-Based Scalability and Collaboration

4.2.5. Augmented Reality (AR) for Field Assistance

4.2.6. Blockchain for Data Security and Traceability

4.2.7. Expansion to Multi-Crop and Global Use Cases

4.2.8. Gamification and Community Engagement

4.2.9. Ethical AI and Sustainability Impact Analysis

4.2.10. Integration with Agricultural Supply Chains

4.2.11. Teleconsultation with Agri-Experts

4.2.12. Drone-Based Surveillance and Precision Agriculture

4.2.13. Multilingual and Low-Literacy Support

4.2.14. Long-Term Impact Studies

CONCLUSION

87

FUTURESCOPE

88-89

REFERENCES

90

ABSTRACT

A Smart Plant Leaf Disease Detection System leverages advanced technologies such as image processing, machine learning, and IoT sensors to automatically detect and diagnose plant diseases from leaf images. The system typically includes a database for storing leaf images, sensor data, and disease information, which is used to train and deploy machine learning models for accurate disease identification. The user interface, often presented as a dashboard, provides real-time monitoring, analysis, and recommendations for disease management, enabling farmers and agriculturists to take timely and effective actions to protect crops and optimize agricultural productivity. This system enhances precision agriculture by reducing the reliance on manual inspection and improving the accuracy and efficiency of disease detection.

CHAPTER 01: Software Requirement Specification

1.1 Introduction

1.1.1 Project Description

Plant health is critical to the agricultural industry and the global economy. The early detection and diagnosis of plant diseases can significantly enhance crop yield and quality. Traditionally, plant disease detection has been performed manually by experts, which is time-consuming, labor-intensive, and prone to human error. In recent years, advancements in technology have paved the way for automated systems that leverage machine learning and image processing techniques to identify plant diseases with high accuracy.

The Smart Plant Leaf Disease Detection System is an innovative solution designed to address these challenges. By utilizing advanced algorithms and artificial intelligence, this system can accurately detect and diagnose various plant diseases from leaf images, providing farmers and agricultural experts with timely information to take appropriate actions.

1.1.2 Purpose:

The Smart Plant Leaf Disease Detection System aims to revolutionize the way plant diseases are detected and managed. Here are the key purposes of this system:

Early Detection of Diseases:

Identify plant diseases at an early stage, allowing for timely intervention and management to prevent widespread damage.

Increased Accuracy:

Utilize advanced machine learning and image processing techniques to accurately diagnose a variety of plant diseases, reducing the likelihood of misdiagnosis.

Efficiency and Speed:

Provide rapid disease detection compared to traditional manual methods, saving time and labor for farmers and agricultural professionals.

Cost Reduction:

Decrease the costs associated with plant disease management by reducing the need for expert consultation and minimizing crop losses through early detection.

Enhanced Accessibility:

Develop user-friendly interfaces and mobile applications to make disease detection technology accessible to a wide range of users, including small-scale farmers and agricultural workers.

Support Sustainable Agriculture:

Promote sustainable farming practices by enabling precise and timely interventions, thereby reducing the reliance on chemical treatments and minimizing environmental impact.

Data-Driven Insights:

Collect and analyze data on plant diseases to provide valuable insights and trends, helping farmers make informed decisions and improve crop management strategies.

Improved Crop Yield and Quality:

By effectively managing plant health, the system aims to enhance overall crop yield and quality, contributing to food security and economic stability for farmers.

1.1.3 Scope:

The future of the Smart Plant Leaf Disease Detection System is bright, with numerous opportunities for advancement and expanded applications. Here are several key areas where the Future scope of this technology can be envisioned:

Expansion to Other Plant Parts:

Comprehensive Diagnostics: Extend the technology to detect diseases in stems, fruits, roots, and flowers, providing a more comprehensive plant health monitoring solution.

Inclusion of More Plant Species:

Diverse Crop Coverage: Expand the system to cover a wider variety of plant species and crops, benefiting a broader range of agricultural sectors.

Enhanced Machine Learning Models:

Improved Accuracy: Develop more sophisticated machine learning algorithms to enhance the accuracy and robustness of disease detection, even under varying environmental conditions.

Adaptive Learning: Implement adaptive learning techniques that allow the system to continuously improve and update its models based on new data.

Cloud-Based Data Management:

Scalable Storage: Utilize cloud computing for scalable storage and processing of large volumes of data, facilitating better data analysis and sharing.

Collaborative Research: Enable collaboration among researchers and agricultural experts worldwide through cloud-based platforms.

Mobile and Web Applications:

User-Friendly Interfaces: Develop intuitive mobile and web applications that allow farmers to easily capture and upload images, receive instant diagnostics, and access disease management recommendations.

1.1.4 Overview:

The Smart Plant Leaf Disease Detection System aims to leverage advanced technologies such as image processing, machine learning to detect plant diseases early and accurately.

The system provides farmers and agricultural professionals with tools for diagnosing and managing plant health, thereby improving crop yields and reducing economic losses.

Stakeholders

Farmers and Gardeners: Primary users who will use the system to monitor and manage plant health.

Agricultural Experts: Professionals who can provide input for model training and validate system accuracy.

Researchers: Individuals focused on improving the algorithms and expanding the system's capabilities.

Developers and Engineers: Technical team responsible for building and maintaining the system.

Policy Makers: Government and regulatory bodies interested in agricultural productivity and food security.

1.1.5 Key features

Image Acquisition

Capture high-resolution images of plant leaves using cameras or mobile devices.

Ensure images are taken in consistent lighting conditions to improve accuracy.

Image Preprocessing

Apply image preprocessing techniques such as noise reduction, contrast adjustment, and segmentation to enhance image quality.

Extract features relevant to disease identification, such as color, texture, and shape.

Disease Detection

Develop machine learning models, particularly Convolutional Neural Networks (CNNs), to classify images and identify diseases.

Train models on a comprehensive dataset of plant leaf images with labeled diseases.

Real-Time Analysis

Integrate with IoT devices (e.g., smart cameras) for real-time data collection and analysis.

Provide instant feedback through a user-friendly interface, such as a mobile app or web application.

Decision Support

Offer actionable recommendations for disease management, including treatment options and preventive measures.

Maintain a knowledge base with information on various plant diseases and best practices for disease control.

Scalability and Accessibility

Design the system to handle large datasets and accommodate a growing number of users.

Ensure accessibility for farmers with varying levels of technical expertise, including those in remote areas.

Non-Functional Requirements

Performance

Ensure the system provides quick and accurate disease detection.

Optimize algorithms for efficient processing and low latency in real-time scenarios.

Disease Detection

Develop machine learning models, particularly Convolutional Neural Networks (CNNs), to classify images and identify diseases.

Train models on a comprehensive dataset of plant leaf images with labeled diseases.

Real-Time Analysis

Integrate with IoT devices (e.g., smart cameras) for real-time data collection and analysis.

Provide instant feedback through a user-friendly interface, such as a mobile app or web application.

Decision Support

Offer actionable recommendations for disease management, including treatment options and preventive measures.

Maintain a knowledge base with information on various plant diseases and best practices for disease control.

Scalability and Accessibility

Design the system to handle large datasets and accommodate a growing number of users.

Ensure accessibility for farmers with varying levels of technical expertise, including those in remote areas.

Non-Functional Requirements

Performance

Ensure the system provides quick and accurate disease detection.

Optimize algorithms for efficient processing and low latency in real-time scenarios.

Reliability

Develop robust models that provide consistent results across different environments and conditions.

Implement error-handling mechanisms and failover processes to maintain system reliability.

Usability

Design an intuitive user interface that is easy to navigate and use.

Provide clear instructions and support to help users effectively utilize the system.

Security

Implement data encryption and secure access controls to protect user data and privacy.

Ensure the system complies with relevant data protection regulations.

Maintainability

Design the system with modular architecture to facilitate updates and maintenance.

Provide documentation and support for ongoing development and improvement.

1.1.6 System Architecture: Hardware Components

Cameras/Smartphones: For capturing high-resolution images of plant leaves.

Software Components

Image Processing Module: Handles image preprocessing and feature extraction.

Machine Learning Module: Trained models for disease detection and classification.

Database: Stores images, disease information, and user data.

User Interface: Mobile app or web application for user interaction and feedback.

Data Flow

Image Capture: Users take pictures of plant leaves using cameras or smartphones.

Image Upload: Images are uploaded to the system, either locally or through IoT devices.

Preprocessing: Images are preprocessed to enhance quality and extract relevant features.

Disease Detection: Machine learning models analyze the images to detect and classify diseases.

Feedback and Recommendations: Results are displayed to the user along with

actionable recommendations for disease management.

Technology Stack

Programming Languages

Python (for machine learning and image processing)

JavaScript (for web application development)

Java/Kotlin (for mobile app development)

Frameworks and Libraries

TensorFlow/PyTorch (machine learning)

OpenCV (image processing)

React/Angular (web development)

Flutter/React Native (mobile app development)

Platforms and Tools

AWS/Azure/GCP (cloud services for scalability and storage)

Docker/Kubernetes (containerization and orchestration)

Git/GitHub (version control and collaboration)

Potential Challenges

Data Quality

Ensuring high-quality images with consistent lighting and clarity for accurate disease detection.

Handling variability in image quality and environmental conditions.

Model Accuracy

Training models to recognize a wide range of diseases with high accuracy.

Continuously updating models with new data to improve performance.

User Adoption

Educating users on the benefits and usage of the system.

Addressing potential resistance to adopting new technology, especially in traditional farming communities.

Future Enhancements

Expansion of Disease Database: Continuously update the system with new diseases and symptoms.

Integration with Drones: Use drones for large-scale monitoring and data collection.

Predictive Analytics: Implement predictive models to forecast disease outbreaks based on environmental data and historical patterns.

Community Support: Develop a platform for users to share experiences and best practices.

1.1.7 Glossary:

TERMS	DEFINITION
Admin	Admin panel for a plant leaf disease detection system involves creating a robust and secure interface that allows the admin to manage user accounts, monitor system usage, and ensure the integrity of the system. Below are the key functionalities and steps you might include:
Login Page	The login system will be a E-mail-based login system. Where the E-mail will be the profile id. This process will be the same for admin and users. Login through mail OTP:
Database	In a Smart Plant Leaf Disease Detection System, databases store images, metadata, models, user information, and disease details. They typically use SQL, NoSQL, and cloud storage to ensure efficient data handling for accurate disease detection.
Editor	Not explicitly defined in this SRS, but generally, this would be a role responsible for reviewing and managing content or data within the system
Real-Time Data Monitoring	This involves continuously collecting and analyzing data from plant leaves using sensors, cameras, or other devices to detect diseases as they occur. It enables quick identification of issues like fungal infections, bacterial spots, or nutrient deficiencies, allowing for timely intervention to protect crops.
Development Team	This group consists of engineers, programmers, and domain experts who design and build the plant leaf disease detection system. They create the software algorithms, integrate hardware components, and ensure the system is user-friendly and accurate in diagnosing plant health.
User	In a Smart Plant Leaf Disease Detection System, the user interacts with databases storing leaf images, sensor data, models, and disease information. These systems use SQL, NoSQL, and cloud storage for accurate and efficient disease detection.
Dashboard	The dashboard in a Smart Plant Leaf Disease Detection System displays real-time data, including leaf images, detected diseases, and sensor readings, while providing insights and actionable recommendations for plant health management.

1.2 Requirement Analysis

Software Specifications:

1.2.1 Operating System:

- Windows 11 laptop/system

1.2.2 Development Tools:

- **IDE/Editor:** Visual Studio Code or STS
- **Version Control:** Git and GitHub

1.2.3 Backend Technologies:

- **Programming Language:** Java, Node.js
- **Framework:** Angular
- **Database:** MySQL
- **Server:** Apache Tomcat

1.2.4 Frontend Technologies:

- **Languages:** HTML, CSS, JavaScript
- **Package Manager:** npm (Node Package Manager)

1.2.5 Machine Learning Technologies:

Flask FrameWork
Image Acquisition
Preprocessing
Feature Extraction
Classification
Model Optimization
Deployment
Convolutional Neural Networks (CNNs)
Transfer Learning

1.2.6 APIs and Protocols:

- **RESTful APIs:** For communication between frontend and backend

1.2.7 Communication Protocol

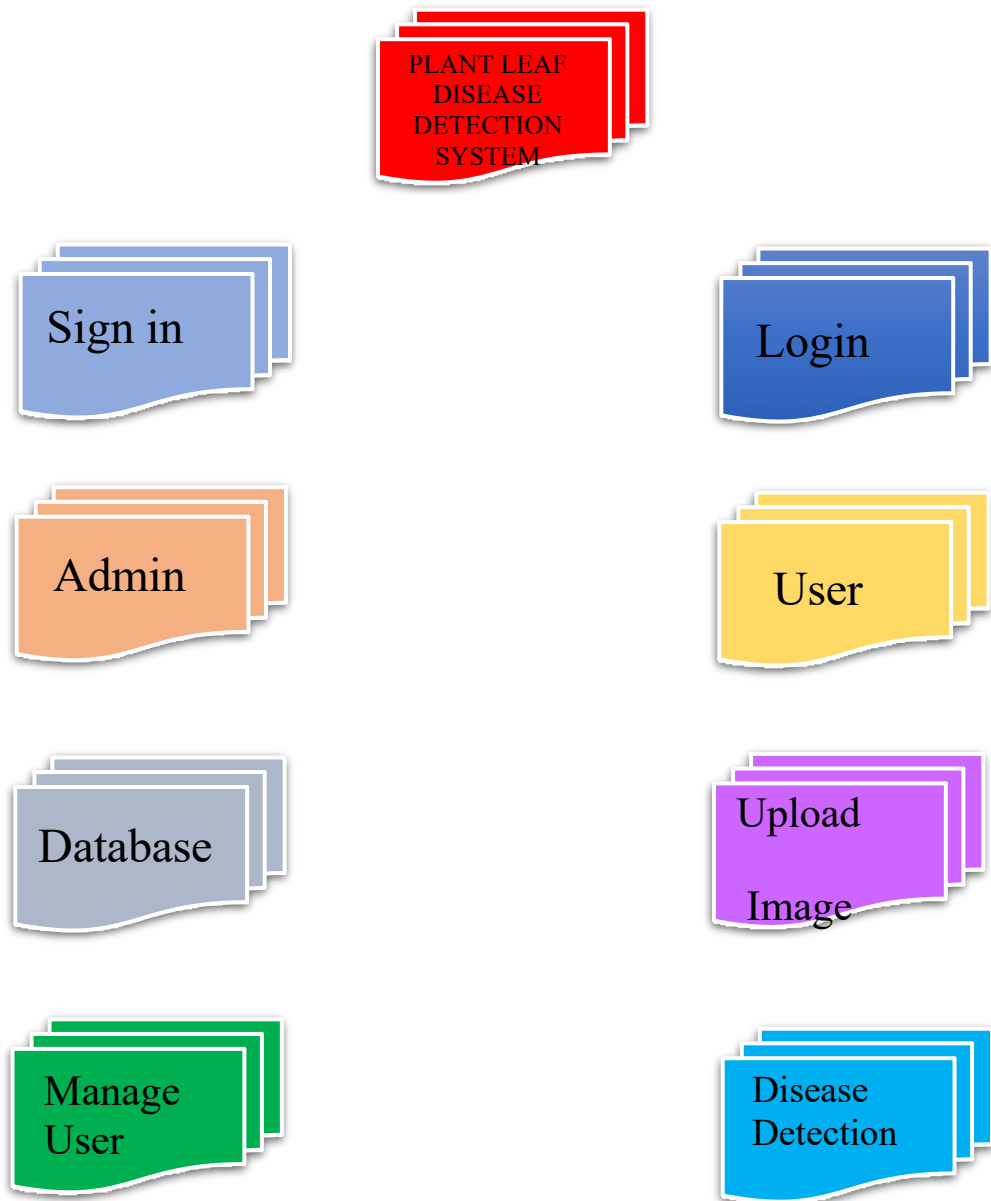
- **TCP/IP :** Ensures reliable data transfer between devices, often used for streaming leaf images or model outputs in networked systems.

- **HTTPS:** To handle the signalling between clients and the server, session management, user authentication, and the transfer of metadata.
- **Web-Sockets :** To provide a persistent, full-duplex communication channel between clients and servers, particularly for real-time signalling and chat.

1.2.8 Testing Tools:

- **Unit Testing:** JUnit for Java
- **Integration Testing:** Postman for API testing

MAIN BLOCKS OF PLANT LEAF DISEASE DETECTION SYSTEM



1.3 Modules

Modules and Functionalities

There are two actors of the application who interact directly with the system: Admin, T&P, and User, who are indirect actors of the application.

Register Form

The Administrator must register by submitting all required documents, which they will then verify. Only after thorough verification will users be allowed to register in the system. These documents will also be maintained by the Administrator in the database for future reference, particularly concerning the plant leaf diseases system.

Login Form:-

The login system will be a E-mail-based login system. Where the E-mail will be the profile id. This process will be the same for admin and users.

Login through mail OTP:

Admin:-

admin panel for a plant leaf disease detection system involves creating a robust and secure interface that allows the admin to manage user accounts, monitor system usage, and ensure the integrity of the system. Below are the key functionalities and steps you might include:

Admin Authentication

- **Admin Login:** Ensure that only authorized personnel can access the admin panel by implementing an admin login system.
- **Role-Based Access Control (RBAC):** Distinguish between different user roles (e.g., Admin, Regular User, Super Admin) and restrict access to sensitive parts of the system based on roles.

User Management

View Users:

Admins should be able to view a list of all users.

- Display relevant information such as username, email, registration date, last login, and role.

Add Users:

- Allow admins to manually add new users to the system.
- Admins can assign roles, set temporary passwords, and notify users via email.

Edit Users:

- Enable admins to edit user details, including roles, email, and account status.
- Implement role changes, password resets, or any other necessary modifications.

Delete/Deactivate Users:

- Allow admins to deactivate or delete user accounts.
- Deactivating should make the account inactive without deleting data, whereas deleting should remove all user data.
- Consider implementing soft deletion, where a user is marked as deleted but data remains recoverable.

Reset Passwords:

- Provide functionality for admins to reset user passwords.
- Send an email to the user with a link to reset their password securely.

3. User Activity Monitoring

• **Login History:**

- Track and display user login history, including IP addresses, timestamps, and login status (successful/failed).

• **User Actions:**

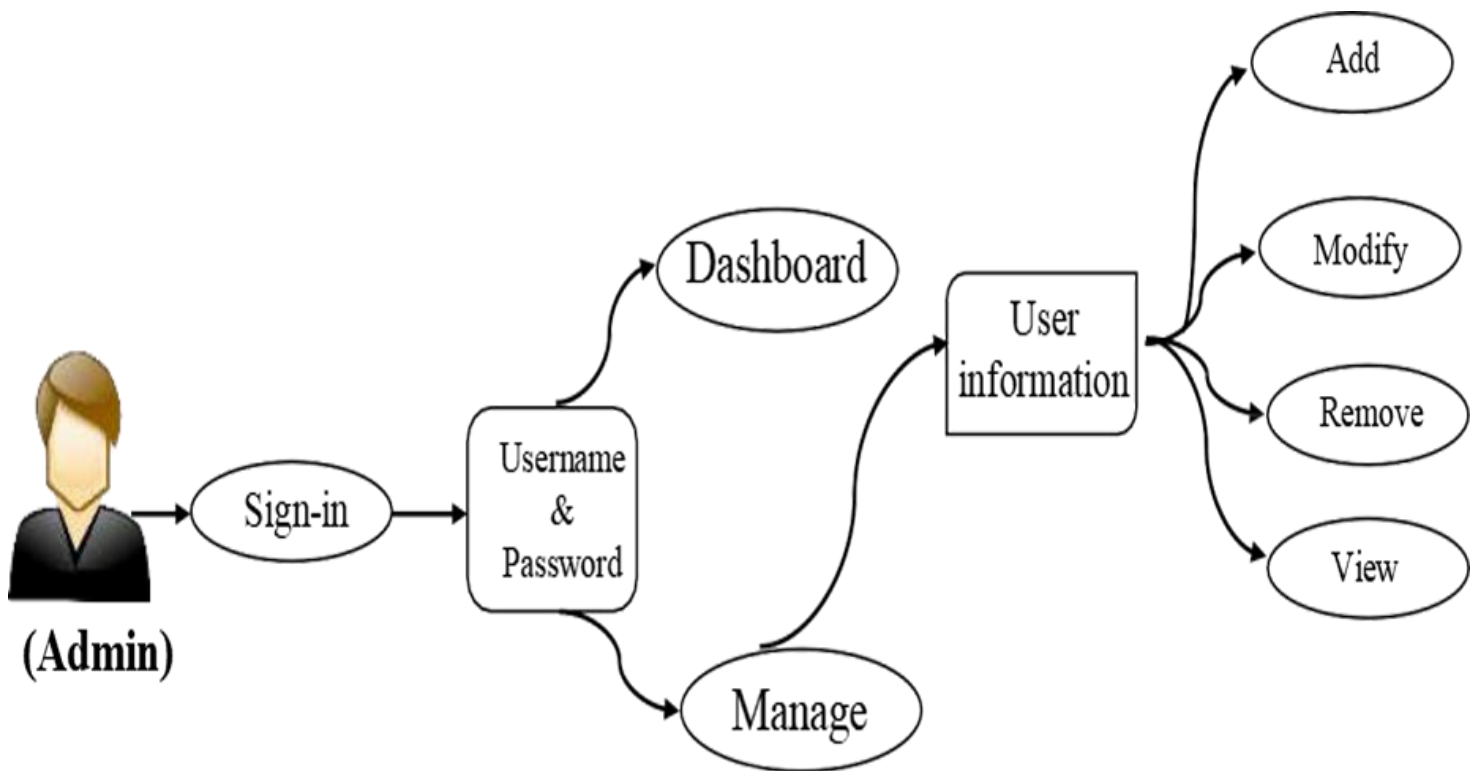
- Monitor actions performed by users within the system, such as uploading images for disease detection, viewing reports, or changing settings.
- Generate logs for these activities to ensure accountability.

1.4. Use Case Diagram

1.4.1 Use Case Diagram For Admin

Administrator : Use Case Diagram

The System shall allow the administrator to add, modify & remove user information equipment of following fields: NAME/ID, PASSWORD/SECRET CODE, EMAIL ADDRESS, USERS, PRIORITY, VIRTUAL LOCATION, and DESCRIPTION .

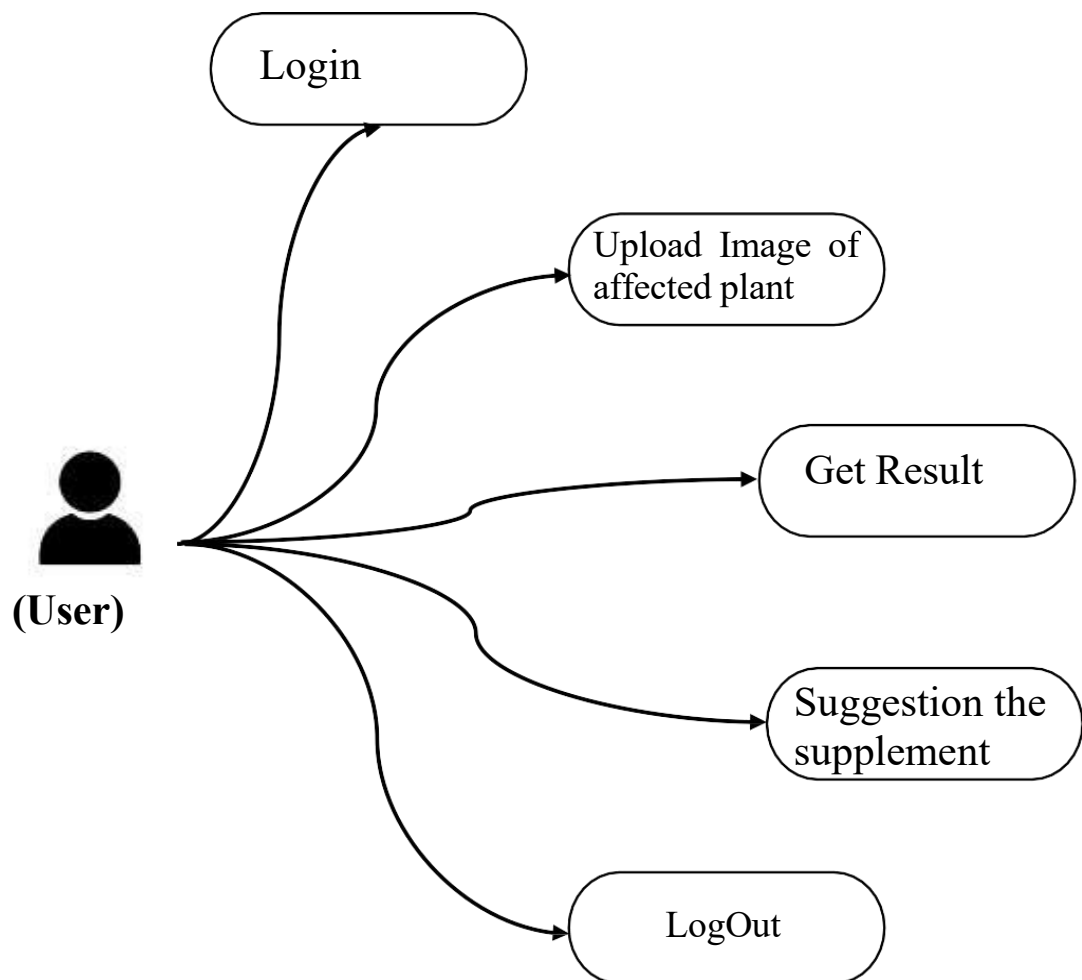


1.4.2 Use Case Diagram For Faculty

The faculty initiates the video-conferencing session, sets permissions for participants, and adjusts settings (e.g., video, microphone access).

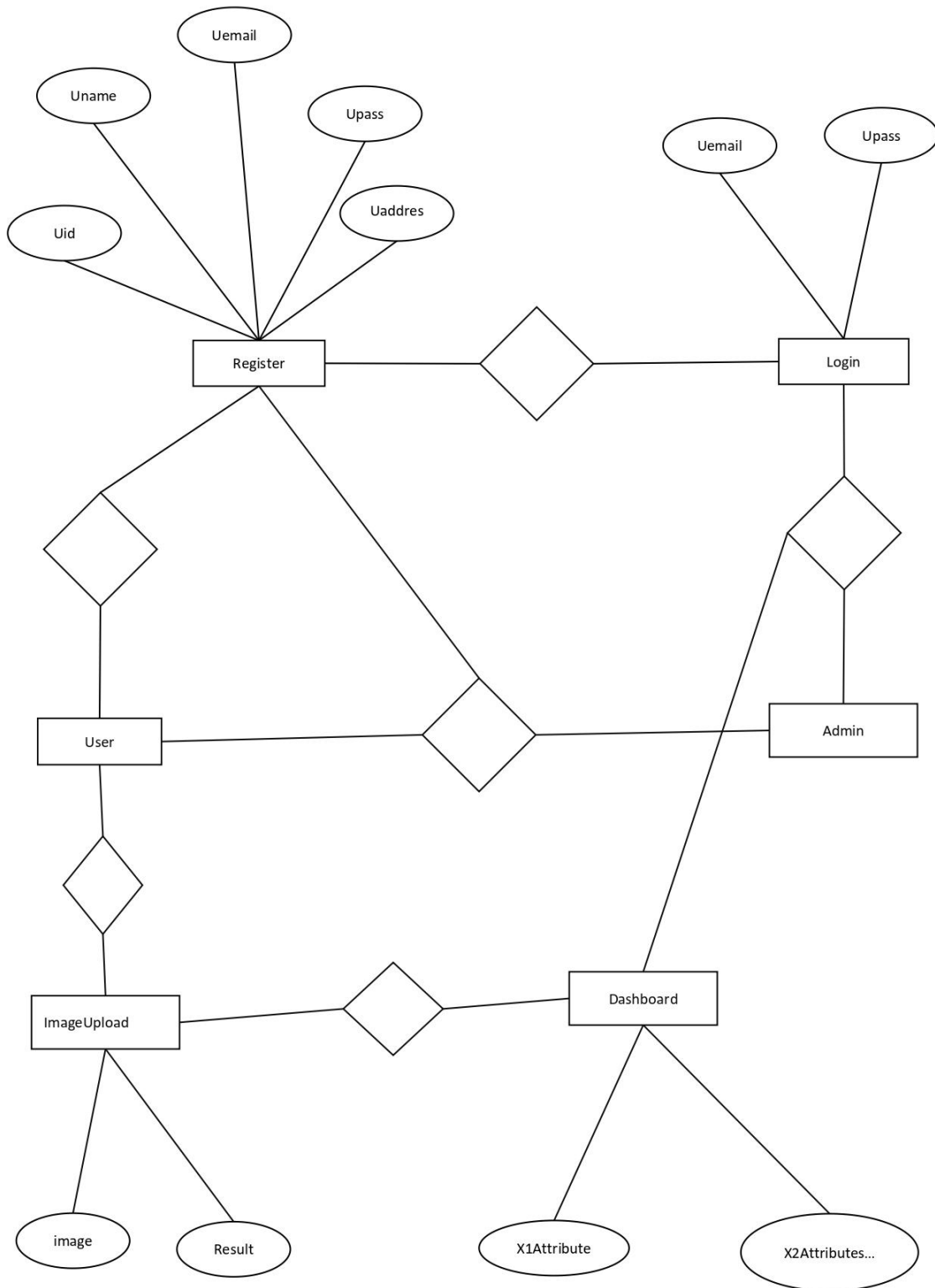
The faculty shares presentation content, videos, or other materials relevant to the session topic. The faculty manages participant interactions, including enabling/disabling screen sharing, muting participants, and setting up breakout rooms.

The faculty reviews attendance during or after the session and notes which participants have left or joined. Upon session completion, the faculty ends the session, and attendance and engagement logs are stored for future access.



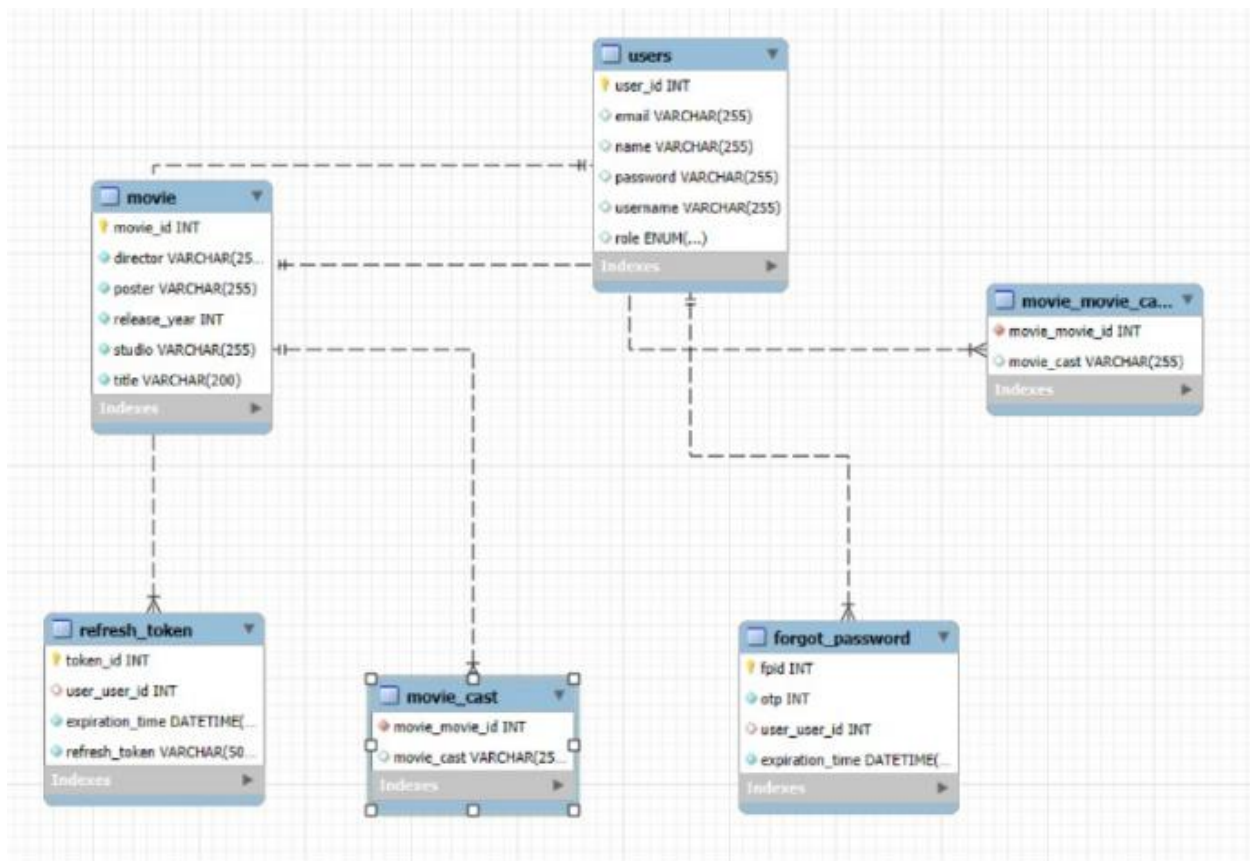
CHAPTER 02: Design and Analysis

2.1 E.R Diagram:



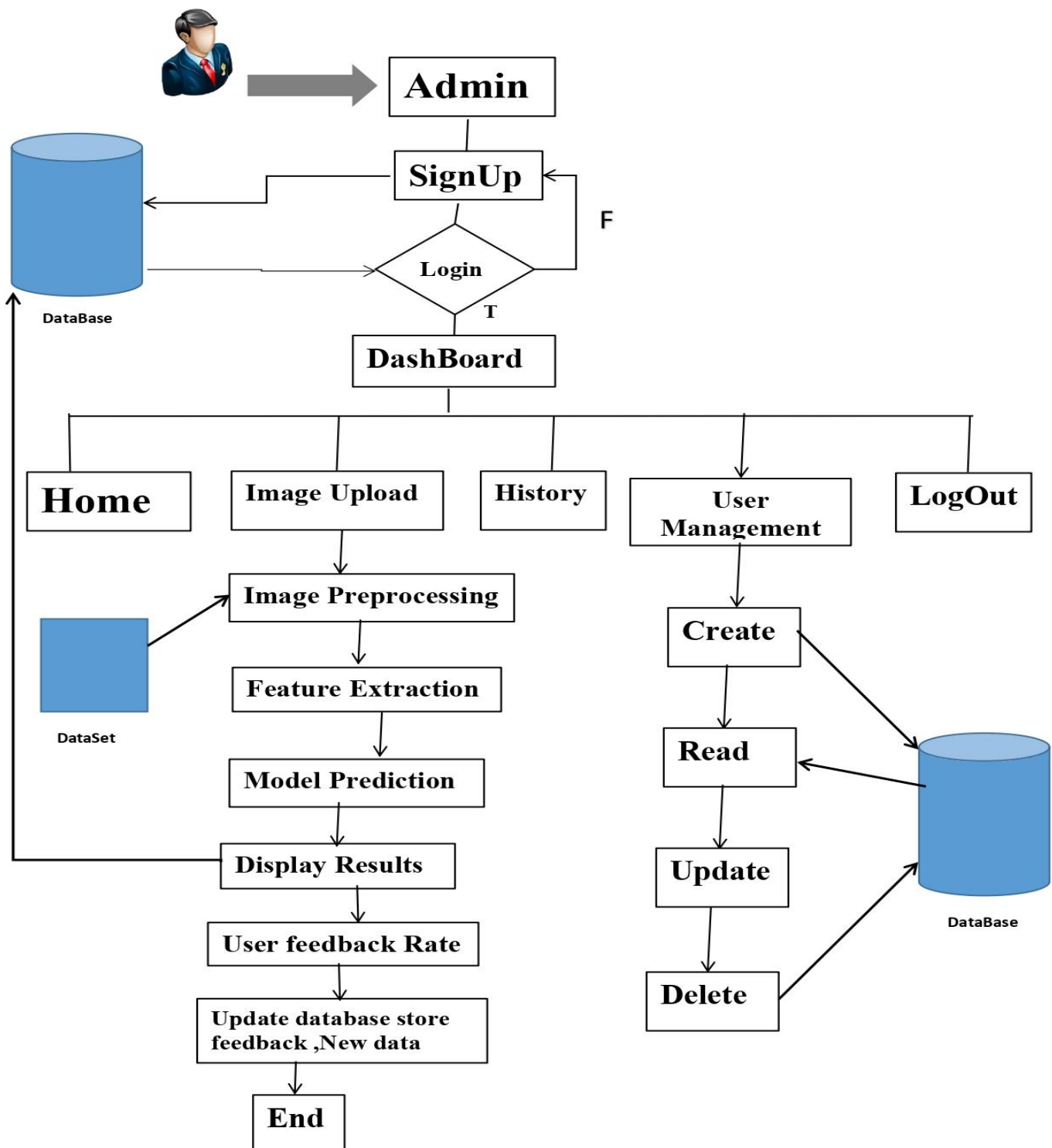
2.2 Schema Diagram

A Schema Diagram is a visual representation of a database's structure, showcasing tables, attributes (columns), and relationships between them. It helps in understanding how data is organized and how different entities interact within a system.

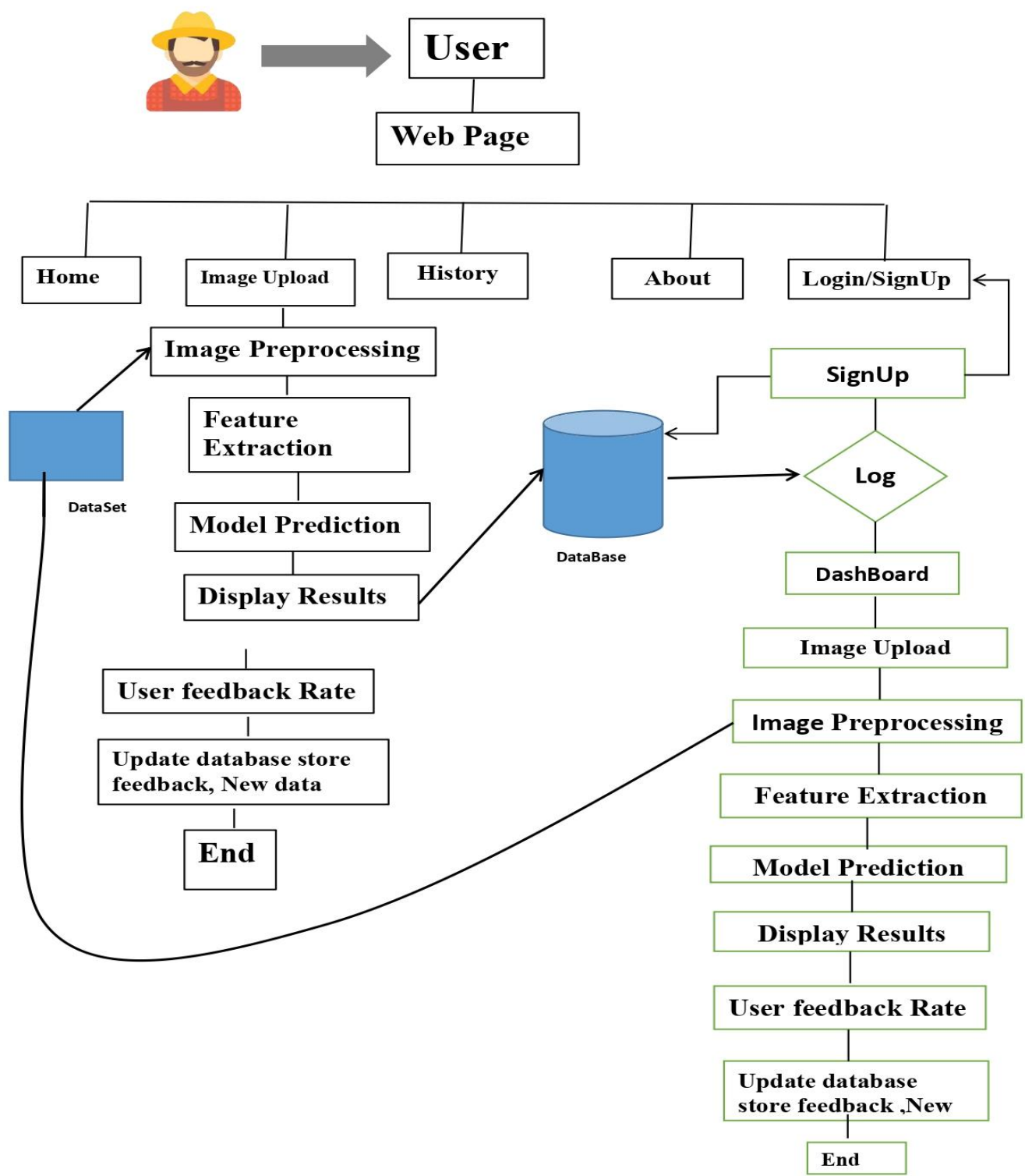


2.3 Activity Flow Diagram

ACTIVATE FLOW DIAGRAM : ADMIN



ACTIVATE FLOW DIAGRAM : USER (STUDENTS/FACULTY)



Overview

This Software Requirements Specification (SRS) document provides detailed use case diagrams along with their descriptions to illustrate system interactions and functionalities. It outlines the essential functional and non-functional requirements, ensuring a comprehensive understanding of the system's capabilities. The document serves as a blueprint for system development, helping stakeholders, developers, and testers align on expectations. By detailing various use cases, it clarifies user interactions, system behaviour, and dependencies, ensuring efficient implementation and usability. This structured approach enhances the clarity, maintainability, and scalability of the software system.

1. MICROPHONE & CHER UI COMPONENTS MODULES Tables In Database

Result Grid		Filter Rows:		Edit:		Export/Imports:		Wrap Cell Contents:	
id	city	email	name	password	role				
5	test4	test4@gmail.com	Test4	\$2a\$10\$2sk7tQvzk/EYRztG7bhCujVGg0oZ0amz...	USER				
6	test5	test5@gmail.com	Test5	\$2a\$10\$b258RFLUOZPyuRhAsIw0NI0bS1B.upE...	ADMIN				
7	test6	test6@gmail.com	Test6	\$2a\$10\$j1.YbL5Sf6MsMt47KmWDOeJaYrogkx...	USER				
8	test7	test7@gmail.com	Test7	\$2a\$10\$wle6XJmLWBf5fP9rATytpOk4iZOmvaM...	ADMIN				
9	test City	test1@gmail.com	Test1	\$2a\$10\$Fzq3f4tfWPHh7..HfvyQ1OJccVsIXsvS...	ADMIN				
10	test2 City	test2@gmail.com	Test2	\$2a\$10\$mqwTc/CkaAcmx.FWakEC2.O5LdKoC79...	USER				
11	TestqCity	test1@gmail.com	test1	\$2a\$10\$N3nVO0behSO7ujKL2M5T2uXnZTecyR...	ADMIN				
12	test2City	test2@gmail.com	test2	\$2a\$10\$QStEv9rAuV8.GfJZaCJegaTJlctzMJa...	USER				
13	test2City	nikhi@gmail.com	test2	\$2a\$10\$BzPdp3ls2b3gRPYPtwjQ1Oo875.4336F...	ADMIN				
14	demo1City	demo1@gmail.com	demo1	\$2a\$10\$75HrunwVklK/NJWfGGuu6MgEdI.bXw...	ADMIN				
15	demo2City	demo2@gmail.com	demo2	\$2a\$10\$BcU2Y77moEjRz4R914QOC8V8oDM...	USER				
16	abc1 city	abc1@gmail.com	abc1	\$2a\$10\$Lu4a/N6JnF0oeB6BqR0vuXAShiteGzV...	ADMIN				
17	abc2city	abc2@gmail.com	abc2	\$2a\$10\$KpJL1MG/k7jWWi6UJ/35.OHyC7/R2V6...	USER				
18	abc21city	abc21@gmail.com	abc21	\$2a\$10\$2RvD9hIPK.ocaM/Yy8E6vOWK6k1o2...	ADMIN				

CHAPTER – 3 PROJECT WORK PART-I

3.1 Front-End Code

3.1.1 Landing Page HTMLM - navbar.component.html

```
<!-- @if (isLoggedIn) { -->

<div class="nav2">
  @if(isAdmin){
    <div class="adminNav">
      <div class="button-gradient"><a routerLink="/home">Home</a></div>
      <div class="manageUser button-gradient "><a routerLink="/users">Manage Users</a></div>
      <button class="button-gradient logoutbtn" (click)="logout()">Logout</button>

    </div>
  }@else if (isUser) {
    <div class="button-gradient"><a routerLink="/home">Home</a></div>
    <div class="button-gradient"><a routerLink="/about">About-US</a></div>
    @if(isLoggedIn){ <button class="button-gradient logoutbtn" (click)="logout()">Logout</button>}
    @else{
      <button class="button-gradient"><a routerLink="/login">Login</a></button>
    }

  } @else {
    <div class="button-gradient"><a routerLink="/home">Home</a></div>
    <div class="button-gradient"><a routerLink="/about">About-US</a></div>
    <div class="button-gradient"><a routerLink="/contact">Contact-US</a></div>
    @if(!isAuthenticated){
      <div class="button-gradient"><a routerLink="/registerpage">Register</a></div>
    }
    <div class="button-gradient"><a routerLink="/login">Login</a></div>

  }

<!-- <div><button (click)="toggleLogin()" routerLink="/login">Login</button></div> -->
```

```

<!-- @if (isLoggedIn) { -->
<!-- } @else { -->
<!-- <div><button class="button-gradient" (click)="logout()">Logout</button></div> -->
<!-- } -->
</div>
<!-- } @else{ -->

```

```

<!-- } -->

```

3.1.2 Landing Page HTMLM - navbar.component.CSS

```

*{
padding: 0;
margin: 0;
}
.link-button {
background-color: blue;
color: rgb(15, 14, 14);
padding: 10px;
border-radius: 5px;

text-decoration: none;
}
/*
nav {
background: rgb(2,0,36);
background: linear-gradient(90deg, rgba(2,0,36,1) 0%, rgba(203,218,222,1) 63%, rgba(0,212,255,1) 100%);

background-color: #12e691;
padding: 10px 0;
padding: 20px;
font-size: 20px;
font-weight: 600;
} */
.nav2 ,.adminManage {

```

```
width:97%;
/* background: rgb(2,0,36);
background: linear-gradient(90deg, rgb(79, 116, 134) 0%, rgba(203,218,222,1) 63%, rgba(0,212,255,1)
  100%);
*/

padding: 10px 0;
padding: 20px;
font-size: 20px;
font-weight: 600;
display: flex;
flex-direction: row;
justify-content: space-between;
}

ul {
list-style-type: none;
padding: 0;
margin: 0;
text-align: center;
}

li {
display: inline-block;
margin: 0 10px;
}

a {
color: #1c1b1b;
text-decoration: none;
padding: 8px 10px;
border-radius: 5px;
transition: background-color 0.3s ease;
}
```

```
@media only screen and (max-width: 600px) {  
nav {  
padding: 5px 0;  
}
```

```
li {  
display: block;  
margin: 10px 0;  
}  
}
```

```
h2{  
text-align: center;  
margin-top: 20px;  
}
```

```
.button-gradient {  
font-size: 22px;
```

```
/* background: linear-gradient(90deg, #01827f, #e0fafb); */
```

```
background: rgb(2,0,36);
```

```
background: linear-gradient(90deg, rgba(2,0,36,1) 0%, rgba(208, 247, 174, 0.95) 0%, rgb(228, 250, 236)  
40%, rgb(220, 251, 233) 99%);
```

```
border: none;
```

```
border-radius: 4px;
```

```
cursor: pointer;
```

```
transition: transform 0.2s ease;
```

```
margin-top: 1px;
```

```
text-align: center;
```

```
color:rgb(15, 15, 15);
```

```
display: flex;
```

```
flex-direction: row;
```

```
justify-content: center;
```

```
align-items: center;
```

```
padding:0px 6px;
```

```
}
```

```
.button-gradient:hover {  
    transform: scale(1.05);  
}
```

```
.logoutbtn{  
    padding: 7px 9px;  
  
}
```

```
.nav2 ,.adminNav{  
    display: flex;  
    justify-content: space-evenly;  
    align-items: center;  
    column-gap: 100px;  
    margin-top: 10px;  
}
```

```
.nav2{  
    background: rgb(2,0,36);  
    background: linear-gradient(90deg, rgb(6, 130, 23) 0%, rgb(229, 247, 220) 99%);  
    width: 97.5%;  
    }
```

3.1.3 Landing Page HTMLM - navbar.component.ts

```
import { Component, OnInit } from '@angular/core';  
import { UserServiceService } from '../Services/user-service.service';  
import { CommonModule, NgIf } from '@angular/common';  
import { Router, RouterLink, RouterOutlet } from '@angular/router';
```

```
@Component({  
    selector: 'app-navbar',  
    standalone: true,  
    imports: [CommonModule, RouterLink, RouterOutlet, NgIf],  
    templateUrl: './navbar.component.html',  
    styleUrls: ['./navbar.component.css']  
})
```

```

export class NavbarComponent implements OnInit {
  constructor(private readonly userService: UserServiceService , private router : Router){}

  isAuthenticated:boolean = false;
  isAdmin:boolean = false;
  isUser:boolean = false;
  isLoggedOut:boolean = true;
  isLoggedIn: boolean = true;

  ngOnInit(): void {
    this.isAuthenticated = this.userService.isAuthenticated();
    this.isAdmin = this.userService.isAdmin();
    this.isUser = this.userService.isUser();
    console.log("isAdmin : " , this.isAdmin);
  }

  logout():void{
    this.userService.logout();
    this.isAuthenticated = false;
    this.isAdmin = false;
    this.isUser = false;
    this.isLoggedIn=true;
    this.router.navigate(['/home']);
    // window.location.reload();
  }

  login(){
    if(this.isLoggedIn){
      this.isLoggedIn =!this.isLoggedIn;
      window.location.reload();
    }
    this.router.navigate(['/login']);
  }

  // Create a toggle button
  toggleLogin(){
    if(this.isLoggedIn){
      this.isLoggedIn =!this.isLoggedIn;
    }
  }
}

```

3.1.4 home.component.html

```

<div class="div1">
  <div class="homeDiv">
    <h2>SMART PLANT LEAF DESEASE DETECTION SYSTEM</h2>
    <h3>Welcome to the Home Page</h3>
  </div>
</div>

```

```

<!-- input image section -->
<div class="inputDiv">
  <div class="inDiv">
    <label for="imageUpload">Upload Image:</label>
    <input type="file" id="imageUpload" name="imageUpload" accept="image/*">
  </div>
  <div class="uploadbtn1"><a routerLink="/result">Upload</a></div>

</div>
</div>
</div>

```

3.1.5 home.component.css

```

.homeDiv {
  text-align: center;
  height: 100vh;
}
h2 {
  text-align: center;
  padding-top: 50px;
  padding-bottom: 20px;
}
h3 {
  padding-bottom: 20px;
}

.inputDiv {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  margin: 20px;
  padding: 70px;
  border: 5px solid #f6f4f4;
  border-radius: 10px;
  /* background-color: #ed5b07; */
  background: rgb(2,0,36);
  background: linear-gradient(90deg, rgb(23, 14, 202) 0%, rgba(199, 250, 154, 0.95) 0%, rgb(52, 248, 2) 40%, rgb(141, 242, 9) 99%);

  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  max-width: 600px;
  margin: 0 auto;
  max-height: 100%;
  /* position: relative; */
  top: 35%;
  left: 35%;
}

label {
  margin-bottom: 10px;
  font-size: 30px;
}

```

```

    font-weight: bold;
    color: #010101;
}

input[type="file"] {
    margin-left: 20px;
    padding: 10px;
    border: 5px solid #f6f2f2;
    border-radius: 10px;
    font-size: 17px;
    cursor: pointer;
    transition: border-color 0.3s;
}

input[type="file"]:hover {
    border-color: #eabef4; /* Change border color on hover */
    box-shadow: inset;
    box-shadow: 0px 0px 50px white;
}

.bgImg img{
    height: 100%;
    width: 100%;
    position: relative;
    top:0;
}

.uploadbtn1 {
    padding: 8px 5px;
    font-size: 16px;
    font-weight: bold;
    color: #beb6b6;
    background: linear-gradient(90deg, #76932b, #f9fcf9);
    border: none;
    border-radius: 10px;
    cursor: pointer;
    transition: transform 0.2s ease;
    margin-top: 15px;
    text-align: center;
    color:rgb(207, 20, 20);
    width: 300px;
}

.uploadbtn1 a{
    color: #050505;
    text-decoration: none;
    font-weight: bold;
    font-size: 22px;
}

```

3.1.6 home.component.ts

```
import { Component } from '@angular/core';
```



```

import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [RouterLink],
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent {

  //
  imageUrl:string='https://images.wallpaperscraft.com/image/single/fern_plant_leaves_139338_3840x2400.
  jpg';

  // Write a logic , when the once the component is loaded a reload only for once time and then stop program
  execute automatically
  constructor() {
    console.log("isReloaded above of the if : " , localStorage.getItem('isReloaded'));

    if (localStorage.getItem('isReloaded') === 'false') {
      localStorage.setItem('isReloaded', 'true');
      // Your code to reload the page or execute any logic here
      // window.location.reload(); // Uncomment this line if you want to reload the page
    }
    console.log("isReloaded above of the if : " , localStorage.getItem('isReloaded'));

    if(localStorage.getItem('isReloaded') === null){
      localStorage.setItem('isReloaded', 'true');
      window.location.reload();
      // localStorage.removeItem('isReloaded');
    }
  }
}

```

3.1.7 register.component.html

```

<div class="container" style="height: 900px;">
<p *ngIf="errorMessage" class="error-message">{{ errorMessage }}</p>
<h2>Add User </h2>
<form (submit)="handleSubmit()">
  <div class="form-group">
    <label>Name:</label>
    <input type="text" name="name" [(ngModel)]="formData.name" required />
  </div>
  <div class="form-group">

```

```

    <label>Email:</label>
    <input type="email" name="email" [(ngModel)]="formData.email" required />
</div>
<div class="form-group">
    <label>Password:</label>
    <input type="password" name="password" [(ngModel)]="formData.password" required />
</div>
<div class="form-group">
    <label>Role:</label>
    <select name="role" [(ngModel)]="formData.role" required>
        <option value="">Select Role</option>
        <option value="ADMIN">ADMIN</option>
        <option value="USER">USER</option>
    </select>
</div>
<div class="form-group">
    <label>City:</label>
    <input type="text" name="city" [(ngModel)]="formData.city" required />
</div>
<button type="submit">Register</button>
</form>
</div>

```

3.1.8 register.component.css

```

.container {
    /* max-width: 500px;
    height: 700px; */
    height: 100%;
    margin: 20px auto 70px;
    padding: 20px;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

.container h2 {
    font-size: 24px;
    margin-bottom: 20px;
    text-align: center;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    font-weight: bold;
}

.form-group input,
.form-group select {
    width: 95%;
}

```

```
padding: 13px;
font-size: 16px;
border: 1px solid #ccc;
border-radius: 5px;
}

button[type="submit"] {
display: block;
width: 100%;
padding: 10px;
font-size: 20px;
color: #fff;
background-color: #007bff;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background-color 0.3s ease;
}

button[type="submit"]:hover {
background-color: #0056b3;
}

.error-message {
color: #dc3545;
font-size: 20px;
margin: 10px;
text-align: center;
}
```

3.1.9 register.component.ts

```
import { Component } from '@angular/core';
import { UserServiceService } from '../Services/user-service.service';
import { Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-register',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './register.component.html',
})
```

```

    styleUrl: './register.component.css'
  })
  export class RegisterComponent {
    formData: any = {
      name: "",
      email: "",
      password: "",
      role: "",
      city: ""
    };
    errorMessage: string = "";

    constructor(
      private readonly userService: UserServiceService,
      private readonly router: Router
    ) {}

    async handleSubmit() {

      // Check if all fields are not empty
      if (!this.formData.name || !this.formData.email || !this.formData.password || !this.formData.role
      || !this.formData.city) {
        this.showError('Please fill in all fields. ');
        return;
      }

      // Confirm registration with user
      const confirmRegistration = confirm('Are you sure you want to register this user?');
      if (!confirmRegistration) {
        return;
      }

      try {
        const token = localStorage.getItem('token');
        if (!token) {
          throw new Error('No token found');
        }

        console.log("This is the register form data : ", this.formData);
        const response = await this.userService.register(this.formData, token);
        console.log("This is the register response data : ", response);
        if (response.statusCode === 200) {
          console.log("Login response data : ", response);
          console.log("Login response data ourUser : ", response.ourUsers);

          this.router.navigate(['/users']);
        } else {
          this.showError(response.message);
        }
      } catch (error: any) {
        this.showError(error.message);
      }
    }
  }

```

```

showError(message: string) {
  this.errorMessage = message;
  setTimeout(() => {
    this.errorMessage = ""; // Clear the error message after the specified duration
  }, 5000);
}
}

```

3.1.10 login.component.html

```

<div class="container">
  <h2>Login</h2>
  <p *ngIf="errorMessage" class="error-message">{{errorMessage}}</p>
  <form (submit)="handleSubmit()">
    <div class="form-group">
      <label for="">Email: </label>
      <input type="text" [(ngModel)]="email" value="" name="email">
    </div>

    <div class="form-group">
      <label for="">Password: </label>
      <input type="password" [(ngModel)]="password" name="password">
    </div>
    <button type="submit">Login</button>

    <div class="registerDiv">
      <a [routerLink]="['/registerpage']"> Don't have an Account ? Register</a>
    </div>

  </form>
</div>

```

3.1.11 login.component.css

```

.container {
  border-radius: 20px;
  max-width: 500px;
  height: 340px;
  margin: 50px auto;
  padding: 20px;
  border: 5px solid #f1f4f5;
  border-radius: 5px;
  box-shadow: 0 2px 5px rgba(118, 116, 116, 0.1);
}

.container h2 {
  font-size: 26px;

```

```
margin-bottom: 10px;
text-align: center;
}
```

```
.form-group {
margin-bottom: 20px;
}
```

```
.form-group label {
display: block;
font-weight: bold;
font-size: 22px;
margin-bottom: 5px;
}
```

```
.form-group input {
width: 95%;
padding: 13px;
font-size: 16px;
border: 1px solid #e8e4e4;
border-radius: 5px;
}
```

```
button[type="submit"] {
display: block;
width: 100%;
padding: 5px;
font-size: 23px;
font-weight: bold;
color: #fff;
background-color: #39a805;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background-color 0.3s ease;
margin-top: 40px;
/* border: 5px solid rgb(148, 145, 145); */
}
```

```
button[type="submit"]:hover {
background-color: #043402;
}
```

```
.error-message {
color: #dc3545;
font-weight: bold;
font-size: 22px;
margin: 10px;
text-align: center;
width: 400px;
height: 50px;
background-color: #f1f8f4;
display: flex;
```

```

    justify-content: center;
    align-items: center;
    padding-left: 50px;
    margin-left: 25px;
    border-radius: 10px;
}

```

```

.registerDiv{
    text-align: center;
    font-weight: bold;
    font-size: 20px;
    margin-top: 30px;
}

```

```

.registerDiv, a{
    text-decoration: none;
}

```

3.1.12 login.component.ts

```

import { Component } from '@angular/core';
import { UserServiceService } from '../Services/user-service.service';
import { Router, RouterLink } from '@angular/router';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

```

```

@Component({
    selector: 'app-login',
    standalone: true,
    imports: [CommonModule, RouterLink, FormsModule],
    templateUrl: './login.component.html',
    styleUrls: ['./login.component.css']
})

```

```

export class LoginComponent {

```

```

    constructor(
        private readonly userService: UserServiceService,
        private router: Router
    ) { }

```

```

    email: string = ""
    password: string = ""
    errorMessage: string = ""
    isLoggedIn: boolean = false;

```

```

    async handleSubmit() {

        if (!this.email || !this.password) {
            this.showError("Email and Password is required");
            return
        }
    }

```

```

try {
  const response = await this.userService.login(this.email, this.password);
  if(response.statusCode === 200){
    localStorage.setItem('token', response.token)
    localStorage.setItem('role', response.role)
    console.log("Role is : ", response.role);

    if(response.role === "USER"){
      this.router.navigate(['/home']);
    } else{
      // reload the window
      this.router.navigate(['/users']);
    }
  } else{
    this.showError(response.message)
  }
} catch (error: any) {
  this.showError(error.message)
}

}

showError(mess: string) {
  this.errorMessage = mess;
  // this.errorMessage = 'Failed your connection during the login operation...';

  setTimeout(() => {
    this.errorMessage = ""
  }, 3000)
}

}

```

3.1.13 userlist.component.html

```

<!-- Content of the table or Content of the admin -->
<div class="container">
  <h2>Management Page</h2>
  <p *ngIf="errorMessage" class="error-message">{{ errorMessage }}</p>
  <button routerLink="/register" class="reg-button">Add User</button>
  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>City</th>
        <th>Actions</th>
      </tr>
    </thead>

```



```

<tbody>
  <tr *ngFor="let user of users" key="user.id">
    <td>{{ user.id }}</td>
    <td>{{ user.name }}</td>
    <td>{{ user.email }}</td>
    <td>{{ user.city }}</td>
    <td>
      <button class="delete-button" (click)="deleteUser(user.id)">Delete</button>
      <button class="update-button" (click)="navigateToUpdate(user.id)">Update</button>
    </td>
  </tr>
</tbody>
</table>
</div>

```

3.1.14 userlist.component.css

```

.container {
  max-width: 80%;
  margin: 30px auto;
  padding: 20px;
  background-color: #d0fffd;
  border-radius: 5px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

```

```

.container h2 {
  font-size: 30px;
  margin-bottom: 20px;
  color: #343a40;
  text-align: center;
}

```

```

.reg-button {
  display: block;
  margin-bottom: 20px;
  padding: 10px 20px;
  font-size: 16px;
  color: #fff;
  background-color: #1b7feb;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

```

```

.reg-button:hover {
  background-color: #0056b3;
}

```

```

table {
  width: 100%;
}

```

```

border-collapse: collapse;
}

table th,
table td {
padding: 8px 12px;
text-align: left;
border-bottom: 1px solid #dee2e6;
color: #050505;
}

table th {
background-color: #9dcbf9;
font-weight: bold;
}

.delete-button,
.update-button {
padding: 9px;
margin: 0px 2px;
font-size: 14px;
color: #fff;
background-color: #dc3545;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background-color 0.3s ease;
}

.update-button {
background-color: #007bff;
}

.delete-button:hover {
background-color: #c82333;
}

.update-button:hover {
background-color: #012750;
}

.error-message {
color: #dc3545;
font-size: 20px;
margin: 10px;
text-align: center;
}

```

3.1.15 userlist.component.ts

```

import { AfterViewInit, Component, OnInit } from '@angular/core';
import { UserServiceService } from '../Services/user-service.service';
import { Router, RouterLink } from '@angular/router';

```

```

import { CommonModule } from '@angular/common';
import { count } from 'rxjs';

@Component({
  selector: 'app-userslist',
  standalone: true,
  imports: [CommonModule, RouterLink],
  templateUrl: './userslist.component.html',
  styleUrls: ['./userslist.component.css']
})
export class UserslistComponent implements OnInit, AfterViewInit {

  users: any[] = [];
  errorMessage: string = "";
  count = 0;
  constructor(
    private readonly userService: UserServiceService,
    private readonly router: Router
  ) {}
  ngAfterViewInit(): void {
    if(this.count==0){
      this.count++;
      // window.location.reload();
    }
  }

  ngOnInit(): void {
    this.loadUsers();
    setTimeout(() => {
      // window.location.reload();
    }, 1000)
    //stop reloading window
    window.location.toString();
  }

  async loadUsers() {
    try {
      const token: any = localStorage.getItem('token');
      const response = await this.userService.getAllUsers(token);
      if (response && response.statusCode === 200 && response.ourUsersList) {
        this.users = response.ourUsersList;
      } else {
        this.showError('No users found.');
```

```
// }  
// write an exapmle if statement
```

```
async deleteUser(userId: string) {  
  const confirmDelete = confirm('Are you sure you want to delete this user?');  
  if (confirmDelete) {  
    try {  
      const token: any = localStorage.getItem('token');  
      await this.userService.deleteUser(userId, token);  
      // Refresh the user list after deletion  
      this.loadUsers();  
    } catch (error: any) {  
      this.showError(error.message);  
    }  
  }  
}
```

```
navigateToUpdate(userId: string) {  
  this.router.navigate(['/update', userId]);  
}  
showError(message: string) {  
  // this.errorMessage = message;  
  this.errorMessage = 'Failed your connection during the fetching users...';  
  setTimeout(() => {  
    this.errorMessage = ""; // Clear the error message after the specified duration  
  }, 5000);  
}
```

```
// Login Logics  
// constructor(private readonly userService: UserServiceService, private router : Router){}  
  
}
```

3.2 BACK-END

3.2.1 SpringBootWithAngular17Application

```
package com.phegondev.usermanagementsystem;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringBootWithAngular17Application {

    public static void main(String[] args) {
        SpringApplication.run(SpringBootWithAngular17Application.class, args);
        System.out.print("\n\t\t\t\t\t Our Spring Boot And Angular17 Smart Plant Leaf Disease Detection
Application is running... ");
    }

}
```

3.2.2 OurUsersMongo

```
package com.phegondev.usermanagementsystem.entity;

import java.util.Collection;
import java.util.List;

import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;
import org.bson.types.ObjectId;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

//import jakarta.persistence.Entity;
//import jakarta.persistence.GeneratedValue;
//import jakarta.persistence.GenerationType;
//import jakarta.persistence.Id;
//import jakarta.persistence.Table;
import lombok.Data;
//
//@Entity
//@Table(name = "ourusers")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class OurUsersMongo implements UserDetails {
```

```

@Id
private ObjectId id;
@Indexed(unique = true)
private String email;
private String name;
private String password;
private String city;
private String role;

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority(role));
}

@Override
public String getUsername() {
    return email;
}

@Override
public boolean isAccountNonExpired() {
    return true;
}

@Override
public boolean isAccountNonLocked() {
    return true;
}

@Override
public boolean isCredentialsNonExpired() {
    return true;
}

@Override
public boolean isEnabled() {
    return true;
}
}

```

3.2.3 UsersMongoRepo

```

package com.phegondev.usermanagementsystem.repository;

import java.util.Optional;

//import org.springframework.data.jpa.repository.JpaRepository;

import com.phegondev.usermanagementsystem.entity.OurUsersMongo;

```

```
import org.bson.types.ObjectId;
import org.springframework.data.mongodb.repository.MongoRepository;

public interface UsersMongoRepo extends MongoRepository<OurUsersMongo, ObjectId> {
    Optional<OurUsersMongo> findByEmail(String email);
}
```

3.2.4 UserManagementController

```
package com.phegondev.usermanagementsystem.controller;

import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import com.phegondev.usermanagementsystem.dto.RegRes;
import com.phegondev.usermanagementsystem.entity.OurUsersMongo;
import com.phegondev.usermanagementsystem.service.UsersManagementService;

@RestController
public class UserManagementController {
    @Autowired
    private UsersManagementService usersManagementService;

    @PostMapping("/auth/register")
    public ResponseEntity<RegRes> regeister(@RequestBody RegRes reg){
        return ResponseEntity.ok(usersManagementService.register(reg));
    }

    @PostMapping("/auth/login")
    public ResponseEntity<RegRes> login(@RequestBody RegRes req){
        return ResponseEntity.ok(usersManagementService.login(req));
    }

    @PostMapping("/auth/refresh")
    public ResponseEntity<RegRes> refreshToken(@RequestBody RegRes req){
        return ResponseEntity.ok(usersManagementService.refreshToken(req));
    }

    @GetMapping("/admin/get-all-users")
    public ResponseEntity<RegRes> getAllUsers(){
        return ResponseEntity.ok(usersManagementService.getAllUsers());
    }
}
```

```

    }

    @GetMapping("/admin/get-users/{userId}")
    public ResponseEntity<ReqRes> getUserById(@PathVariable ObjectId userId){
        return ResponseEntity.ok(usersManagementService.getUsersById(userId));
    }

    @PutMapping("/admin/update/{userId}")
    public ResponseEntity<ReqRes> updateUser(@PathVariable ObjectId userId, @RequestBody
    OurUsersMongo reqres){
        return ResponseEntity.ok(usersManagementService.updateUser(userId, reqres));
    }

    @GetMapping("/adminuser/get-profile")
    public ResponseEntity<ReqRes> getMyProfile(){
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        String email = authentication.getName();
        ReqRes response = usersManagementService.getMyInfo(email);
        return ResponseEntity.status(response.getStatusCode()).body(response);
    }

    @DeleteMapping("/admin/delete/{userId}")
    public ResponseEntity<ReqRes> deleteUser(@PathVariable ObjectId userId){
        return ResponseEntity.ok(usersManagementService.deleteUser(userId));
    }
}

```

3.2.5 UsersManagementService

```

package com.phegondev.usermanagementsystem.service;

import com.phegondev.usermanagementsystem.dto.ReqRes;
import com.phegondev.usermanagementsystem.entity.OurUsersMongo;
import com.phegondev.usermanagementsystem.repository.UsersMongoRepo;
import java.util.HashMap;
import java.util.List;
import java.util.Optional;

import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class UsersManagementService {

    @Autowired

```



```

private UsersMongoRepo userRepo; // Keeping the name as userRepo
@Autowired
private JWTUtils jwtUtils;
@Autowired
private AuthenticationManager authenticationManager;
@Autowired
private PasswordEncoder passwordEncoder;

public ReqRes register(ReqRes registrationRequest){
    ReqRes resp = new ReqRes();

    try {
        OurUsersMongo ourUser = new OurUsersMongo(); // Keeping the name as ourUser
        ourUser.setEmail(registrationRequest.getEmail());
        ourUser.setCity(registrationRequest.getCity());
        ourUser.setRole(registrationRequest.getRole());
        ourUser.setName(registrationRequest.getName());
        ourUser.setPassword(passwordEncoder.encode(registrationRequest.getPassword()));
        OurUsersMongo ourUsersResult = userRepo.save(ourUser);
        if (ourUsersResult != null && ourUsersResult.getId() != null) {
            resp.setOurUsers(ourUsersResult);
            resp.setMessage("User Saved Successfully");
            resp.setStatusCode(200);
        } else {
            resp.setStatusCode(500);
            resp.setError("Failed to save user.");
        }
    }

    } catch (Exception e){
        resp.setStatusCode(500);
        resp.setError(e.getMessage());
    }
    return resp;
}

public ReqRes login(ReqRes loginRequest){
    ReqRes response = new ReqRes();
    try {
        authenticationManager
            .authenticate(new UsernamePasswordAuthenticationToken(loginRequest.getEmail(),
                loginRequest.getPassword()));
        OurUsersMongo user = userRepo.findByEmail(loginRequest.getEmail()).orElseThrow();
        var jwt = jwtUtils.generateToken(user);
        var refreshToken = jwtUtils.generateRefreshToken(new HashMap<>(), user);
        response.setStatusCode(200);
        response.setToken(jwt);
        response.setRole(user.getRole());
        response.setRefreshToken(refreshToken);
        response.setExpirationTime("24Hrs");
        response.setMessage("Successfully Logged In");

    } catch (Exception e){

```

```

        response.setStatusCode(500);
        response.setMessage(e.getMessage());
    }
    return response;
}

public ReqRes refreshToken(ReqRes refreshTokenRequest){
    ReqRes response = new ReqRes();
    try{
        String ourEmail = jwtUtils.extractUsername(refreshTokenRequest.getToken());
        OurUsersMongo users = userRepo.findByEmail(ourEmail).orElseThrow();
        if (jwtUtils.isTokenValid(refreshTokenRequest.getToken(), users)) {
            var jwt = jwtUtils.generateToken(users);
            response.setStatusCode(200);
            response.setToken(jwt);
            response.setRefreshToken(refreshTokenRequest.getToken());
            response.setExpirationTime("24Hr");
            response.setMessage("Successfully Refreshed Token");
        }
        response.setStatusCode(200);
        return response;

    } catch (Exception e){
        response.setStatusCode(500);
        response.setMessage(e.getMessage());
        return response;
    }
}

```

```

public ReqRes getAllUsers() {
    ReqRes reqRes = new ReqRes();

    try {
        List<OurUsersMongo> result = userRepo.findAll();
        if (!result.isEmpty()) {
            reqRes.setOurUsersList(result);
            reqRes.setStatusCode(200);
            reqRes.setMessage("Successful");
        } else {
            reqRes.setStatusCode(404);
            reqRes.setMessage("No users found");
        }
        return reqRes;
    } catch (Exception e) {
        reqRes.setStatusCode(500);
        reqRes.setMessage("Error occurred: " + e.getMessage());
        return reqRes;
    }
}

```

```

public ReqRes getUsersById(ObjectId id) {
    ReqRes reqRes = new ReqRes();

```

```

    try {
        OurUsersMongo usersById = userRepo.findById(id).orElseThrow(() -> new RuntimeException("User
Not found"));
        reqRes.setOurUsers(usersById);
        reqRes.setStatusCode(200);
        reqRes.setMessage("Users with id " + id + " found successfully");
    } catch (Exception e) {
        reqRes.setStatusCode(500);
        reqRes.setMessage("Error occurred: " + e.getMessage());
    }
    return reqRes;
}

```

```

public ReqRes deleteUser(ObjectId userId) {
    ReqRes reqRes = new ReqRes();
    try {
        Optional<OurUsersMongo> userOptional = userRepo.findById(userId);
        if (userOptional.isPresent()) {
            userRepo.deleteById(userId);
            reqRes.setStatusCode(200);
            reqRes.setMessage("User deleted successfully");
        } else {
            reqRes.setStatusCode(404);
            reqRes.setMessage("User not found for deletion");
        }
    } catch (Exception e) {
        reqRes.setStatusCode(500);
        reqRes.setMessage("Error occurred while deleting user: " + e.getMessage());
    }
    return reqRes;
}

```

```

public ReqRes updateUser(ObjectId userId, OurUsersMongo updatedUserRequest) {
    ReqRes reqRes = new ReqRes();
    try {
        Optional<OurUsersMongo> userOptional = userRepo.findById(userId);
        if (userOptional.isPresent()) {
            OurUsersMongo existingUser = userOptional.get();
            existingUser.setEmail(updatedUserRequest.getEmail());
            existingUser.setName(updatedUserRequest.getName());
            existingUser.setCity(updatedUserRequest.getCity());
            existingUser.setRole(updatedUserRequest.getRole());

            // Check if password is present in the request
            if (updatedUserRequest.getPassword() != null && !updatedUserRequest.getPassword().isEmpty()) {
                // Encode the password and update it
                existingUser.setPassword(passwordEncoder.encode(updatedUserRequest.getPassword()));
            }

            OurUsersMongo savedUser = userRepo.save(existingUser);
            reqRes.setOurUsers(savedUser);
            reqRes.setStatusCode(200);
            reqRes.setMessage("User updated successfully");
        }
    }
}

```

```

    } else {
        reqRes.setStatusCode(404);
        reqRes.setMessage("User not found for update");
    }
} catch (Exception e) {
    reqRes.setStatusCode(500);
    reqRes.setMessage("Error occurred while updating user: " + e.getMessage());
}
return reqRes;
}

```

```

public ReqRes getMyInfo(String email){
    ReqRes reqRes = new ReqRes();
    try {
        Optional<OurUsersMongo> userOptional = userRepo.findByEmail(email);
        if (userOptional.isPresent()) {
            reqRes.setOurUsers(userOptional.get());
            reqRes.setStatusCode(200);
            reqRes.setMessage("successful");
        } else {
            reqRes.setStatusCode(404);
            reqRes.setMessage("User not found");
        }
    } catch (Exception e){
        reqRes.setStatusCode(500);
        reqRes.setMessage("Error occurred while getting user info: " + e.getMessage());
    }
    return reqRes;
}
}

```

3.2.6 OurUserDetailsService

```
package com.phegondev.usermanagementsystem.service;
```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

```

```
import com.phegondev.usermanagementsystem.repository.UsersMongoRepo;
```

```
@Service
```

```
public class OurUserDetailsService implements UserDetailsService {
```

```
    @Autowired
```

```
    private UsersMongoRepo userRepo;
```

```
    @Override
```

```
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
```

```

        return userRepo.findByEmail(username).orElseThrow();
    }
}

```

3.2.7 JWTUtils

```
package com.phgondev.usermanagementsystem.service;
```

```
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.function.Function;
```

```
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
```

```
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
```

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
```

```
@Component
```

```
public class JWTUtils {
```

```
    private SecretKey Key;
    private static final long EXPIRATION_TIME = 86400000; //24 hours
```

```
    public JWTUtils(){
        String secreteString =
"843567893696976453275974432697R634976R738467TR678T34865R6834R8763T4783786376645387456
73865783678548735687R3";
        byte[] keyBytes = Base64.getDecoder().decode(secreteString.getBytes(StandardCharsets.UTF_8));
        this.Key = new SecretKeySpec(keyBytes, "HmacSHA256");
    }

```

```
    public String generateToken(UserDetails userDetails){
        return Jwts.builder()
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
            .signWith(Key)
            .compact();
    }

```

```
    public String generateRefreshToken(HashMap<String, Object> claims, UserDetails userDetails){
        return Jwts.builder()
            .claims(claims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
            .signWith(Key)
            .compact();
    }
}

```

```

    public String extractUsername(String token){
        return extractClaims(token, Claims::getSubject);
    }

    private <T> T extractClaims(String token, Function<Claims, T> claimsTFunction){
        return
claimsTFunction.apply(Jwts.parser().verifyWith(Key).build().parseSignedClaims(token).getPayload());
    }

    public boolean isTokenValid(String token, UserDetails userDetails){
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
    }

    public boolean isTokenExpired(String token){
        return extractClaims(token, Claims::getExpiration).before(new Date());
    }
}

```

3.2.8 ReqRes

```

package com.phegondev.usermanagementsystem.dto;

import java.util.List;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonInclude;
import com.phegondev.usermanagementsystem.entity.OurUsersMongo;

import lombok.Data;

@Data
@JsonInclude(JsonInclude.Include.NON_NULL)
@JsonIgnoreProperties(ignoreUnknown = true)
public class ReqRes {

    private int statusCode;
    private String error;
    private String message;
    private String token;
    private String refreshToken;
    private String expirationTime;
    private String name;
    private String city;
    private String role;
    private String email;
    private String password;
    private OurUsersMongo ourUsers;
    private List<OurUsersMongo> ourUsersList;
    public int getStatusCode() {
        return statusCode;
    }
    public void setStatusCode(int statusCode) {
        this.statusCode = statusCode;
    }
}

```

```

}
public String getError() {
    return error;
}
public void setError(String error) {
    this.error = error;
}
public String getMessage() {
    return message;
}
public void setMessage(String message) {
    this.message = message;
}
public String getToken() {
    return token;
}
public void setToken(String token) {
    this.token = token;
}
public String getRefreshToken() {
    return refreshToken;
}
public void setRefreshToken(String refreshToken) {
    this.refreshToken = refreshToken;
}
public String getExpirationTime() {
    return expirationTime;
}
public void setExpirationTime(String expirationTime) {
    this.expirationTime = expirationTime;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public String getCity() {
    return city;
}
public void setCity(String city) {
    this.city = city;
}
public String getRole() {
    return role;
}
public void setRole(String role) {
    this.role = role;
}
public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}

```

```

    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}

public OurUsersMongo getOurUsers() {
    return ourUsers;
}
}
}

```

3.2.9 SecurityConfig

```

package com.phegondev.usermanagementsystem.config;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

import com.phegondev.usermanagementsystem.service.OurUserDetailsService;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private OurUserDetailsService ourUserDetailsService;

    @Autowired
    private JWTAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.csrf(AbstractHttpConfigurer::disable).cors(Customizer.withDefaults())
        .authorizeHttpRequests(request -> request
            .requestMatchers("/auth/**",
                "/public/**").permitAll()

            .requestMatchers("/admin/**").hasAnyAuthority("ADMIN")

```



```

        .requestMatchers("/user/**").hasAnyAuthority("USER")

        .requestMatchers("/adminuser/**").hasAnyAuthority("ADMIN", "USER")
            .anyRequest().authenticated()
            .sessionManagement(manager ->
manager.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter,
UsernamePasswordAuthenticationFilter.class);
        return httpSecurity.build();
    }
@Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider daoAuthenticationProvider = new DaoAuthenticationProvider();
        daoAuthenticationProvider.setUserDetailsService(ourUserDetailsService);
        daoAuthenticationProvider.setPasswordEncoder(passwordEncoder());
        return daoAuthenticationProvider;
    }
@Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

@Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
authenticationConfiguration)
        throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }
}

```

3.2.10 JWTAuthFilter

```

package com.phegondev.usermanagementsystem.config;

import java.io.IOException;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import com.phegondev.usermanagementsystem.service.JWTUtils;
import com.phegondev.usermanagementsystem.service.OurUserDetailsService;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class JWTAuthFilter extends OncePerRequestFilter {

    @Autowired
    private JWTUtils jwtUtils;

```

```

@Autowired
private OurUserDetailsService ourUserDetailsService;

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
filterChain) throws ServletException, IOException {

    final String authHeader = request.getHeader("Authorization");
    final String jwtToken;
    final String userEmail;

    if (authHeader == null || authHeader.isBlank()) {
        filterChain.doFilter(request, response);
        return;
    }

    jwtToken = authHeader.substring(7);
    userEmail = jwtUtils.extractUsername(jwtToken);

    if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails userDetails = ourUserDetailsService.loadUserByUsername(userEmail);

        if (jwtUtils.isTokenValid(jwtToken, userDetails)) {
            SecurityContext securityContext = SecurityContextHolder.createEmptyContext();
            UsernamePasswordAuthenticationToken token = new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities()
            );
            token.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            securityContext.setAuthentication(token);
            SecurityContextHolder.setContext(securityContext);
        }
    }
    filterChain.doFilter(request, response);
}
}

```

3.2.11 CorsConfig

```

package com.phegondev.usermanagementsystem.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer webMvcConfigurer() {
        return new WebMvcConfigurer() {
            @Override

```

```

    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedOrigins("*");
    }
}
}
}

```

3.2.12 application.properties

```

spring.application.name=SpringBootWithAngular17
#spring.datasource.url=jdbc:mysql://localhost:3306/spring_angular17_users_management2
#spring.datasource.username=root
#spring.datasource.password=Mantu@123
#spring.jpa.hibernate.ddl-auto=update
#spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.data.mongodb.uri=mongodb+srv://mantukumar4376:YjBqTtRBxp7_an7@cluster0.i6lvqoh.mongodb.net/plt
spring.data.mongodb.database=Plt
spring.data.mongodb.auto.index-creation=true
server.port=1010

```

3.3.1 MACHINE LEARNING BACK-END

3.3.1 app.py

```
import os
from flask import Flask, redirect, render_template, request
from PIL import Image
import torchvision.transforms.functional as TF
import torch
import numpy as np
import pandas as pd
import CNN # Assuming CNN.py contains the model definition

# Load disease and supplement data
disease_info = pd.read_csv('disease_info.csv', encoding='cp1252')
supplement_info = pd.read_csv('supplement_info.csv', encoding='cp1252')

# Load the trained model
model = CNN.CNN(39) # Assuming 39 classes
model.load_state_dict(torch.load("plant_disease_model_1_latest.pt"))
model.eval()

# Function to make predictions
def prediction(image_path):
    image = Image.open(image_path)
    image = image.resize((224, 224))
    input_data = TF.to_tensor(image)
    input_data = input_data.view((-1, 3, 224, 224))
    output = model(input_data)
    output = output.detach().numpy()
    index = np.argmax(output)
    return index

# Initialize Flask app
app = Flask(__name__)

@app.route('/')
def home_page():
    return render_template('home.html')

@app.route('/contact')
def contact():
    return render_template('contact-us.html')

@app.route('/index')
def ai_engine_page():
    return render_template('index.html')

@app.route('/mobile-device')
def mobile_device_detected_page():
    return render_template('mobile-device.html')

@app.route('/submit', methods=['GET', 'POST'])
def submit():
    if request.method == 'POST':
        image = request.files['image']
```

```

filename = image.filename

# Ensure 'static/uploads' directory exists
upload_folder = 'static/uploads'
if not os.path.exists(upload_folder):
    os.makedirs(upload_folder)

file_path = os.path.join(upload_folder, filename)
image.save(file_path)

print(f"Image saved to: {file_path}")

pred = prediction(file_path)
title = disease_info['disease_name'][pred]
description = disease_info['description'][pred]
prevent = disease_info['Possible Steps'][pred]
image_url = disease_info['image_url'][pred]
supplement_name = supplement_info['supplement name'][pred]
supplement_image_url = supplement_info['supplement image'][pred]
supplement_buy_link = supplement_info['buy link'][pred]

return render_template(
    'submit.html', title=title, desc=description, prevent=prevent,
    image_url=image_url, pred=pred, sname=supplement_name,
    simage=supplement_image_url, buy_link=supplement_buy_link
)

@app.route('/market', methods=['GET', 'POST'])
def market():
    return render_template(
        'market.html', supplement_image=list(supplement_info['supplement image']),
        supplement_name=list(supplement_info['supplement name']),
        disease=list(disease_info['disease_name']),
        buy=list(supplement_info['buy link'])
    )

if __name__ == '__main__':
    # app.run(debug=True)
    app.run(debug=True, port=8080)

```

3.3.2 CNN.py

```
import pandas as pd
import torch.nn as nn

class CNN(nn.Module):
    def __init__(self, K):
        super(CNN, self).__init__()
        self.conv_layers = nn.Sequential(
            # conv1
            nn.Conv2d(in_channels=3, out_channels=32,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.Conv2d(in_channels=32, out_channels=32,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.MaxPool2d(2),
            # conv2
            nn.Conv2d(in_channels=32, out_channels=64,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.Conv2d(in_channels=64, out_channels=64,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(2),
            # conv3
            nn.Conv2d(in_channels=64, out_channels=128,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(128),
            nn.Conv2d(in_channels=128, out_channels=128,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(128),
            nn.MaxPool2d(2),
            # conv4
            nn.Conv2d(in_channels=128, out_channels=256,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(256),
            nn.Conv2d(in_channels=256, out_channels=256,
                      kernel_size=3, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(256),
            nn.MaxPool2d(2),
        )

        self.dense_layers = nn.Sequential(
            nn.Dropout(0.4),
            nn.Linear(50176, 1024),
```

```

nn.ReLU(),
nn.Dropout(0.4),
nn.Linear(1024, K),
)

```

```

def forward(self, X):
    out = self.conv_layers(X)

    # Flatten
    out = out.view(-1, 50176)

    # Fully connected
    out = self.dense_layers(out)

    return out

```

```

idx_to_classes = {0: 'Apple___Apple_scab',
1: 'Apple___Black_rot',
2: 'Apple___Cedar_apple_rust',
3: 'Apple___healthy',
4: 'Background_without_leaves',
5: 'Blueberry___healthy',
6: 'Cherry___Powdery_mildew',
7: 'Cherry___healthy',
8: 'Corn___Cercospora_leaf_spot Gray_leaf_spot',
9: 'Corn___Common_rust',
10: 'Corn___Northern_Leaf_Blight',
11: 'Corn___healthy',
12: 'Grape___Black_rot',
13: 'Grape___Esca_(Black_Measles)',
14: 'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)',
15: 'Grape___healthy',
16: 'Orange___Haunglongbing_(Citrus_greening)',
17: 'Peach___Bacterial_spot',
18: 'Peach___healthy',
19: 'Pepper,_bell___Bacterial_spot',
20: 'Pepper,_bell___healthy',
21: 'Potato___Early_blight',
22: 'Potato___Late_blight',
23: 'Potato___healthy',
24: 'Raspberry___healthy',
25: 'Soybean___healthy',
26: 'Squash___Powdery_mildew',
27: 'Strawberry___Leaf_scorch',
28: 'Strawberry___healthy',
29: 'Tomato___Bacterial_spot',
30: 'Tomato___Early_blight',
31: 'Tomato___Late_blight',
32: 'Tomato___Leaf_Mold',
33: 'Tomato___Septoria_leaf_spot',
34: 'Tomato___Spider_mites Two-spotted_spider_mite',
35: 'Tomato___Target_Spot',
36: 'Tomato___Tomato_Yellow_Leaf_Curl_Virus',

```

```
37: 'Tomato___Tomato_mosaic_virus',
38: 'Tomato___healthy'}
```

3.3.3 base.html

```
<!doctype html>
<html lang="en">

<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta3/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-eOJMYsd53ii+scO/bJGFsiCZc+5NDVN2yr8+0RDqr0Ql0h+rP48ckxlpbzKgwra6"
    crossorigin="anonymous">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/ionicons/2.0.1/css/ionicons.min.css">

  <link rel = "stylesheet" href = "https://cdnjs.cloudflare.com/ajax/libs/btn.css/0.2.4/btn.css">

  <title> {% block pagetitle %}

    {% endblock pagetitle %}</title>
  <style>
    .bg {

      background-image: url("images/background.jpeg");

      height: 100%;

      /* Center and scale the image nicely */
      background-position: center;
      background-repeat: no-repeat;
      background-size: cover;
    }

    .file-upload input[type='file'] {
      display: none;
    }

    body {
      background-color: #2a922f;
      /* background: -webkit-linear-gradient(to right, #66a776, #043610);
      background: linear-gradient(to right, #5d9c6a, #059b0d); */
      height: 100vh;
    }

    .rounded-lg {
      border-radius: 1rem;
    }

    .custom-file-label.rounded-pill {
      border-radius: 50rem;
    }
  </style>
```



```
.custom-file-label.rounded-pill::after {  
  border-radius: 0 50rem 50rem 0;  
}
```

```
label {  
  background-color: rgb(2, 46, 15);  
  color: white;  
  padding: 0.5rem;  
  font-family: sans-serif;  
  border-radius: 0.3rem;  
  cursor: pointer;  
  margin-top: 1rem;  
}
```

```
#file-chosen {  
  margin-left: 0.3rem;  
  font-family: sans-serif;  
}
```

```
.footer-basic {  
  padding: 40px 0;  
  background-color: #042908;  
  color: #f1f3f5;  
}
```

```
.footer-basic ul {  
  padding: 0;  
  list-style: none;  
  text-align: center;  
  font-size: 18px;  
  line-height: 1.6;  
  margin-bottom: 0;  
}
```

```
.footer-basic li {  
  padding: 0 10px;  
}
```

```
.footer-basic ul a {  
  color: inherit;  
  text-decoration: none;  
  opacity: 0.8;  
}
```

```
.footer-basic ul a:hover {  
  opacity: 1;  
}
```

```
.footer-basic .social {  
  text-align: center;  
  padding-bottom: 25px;  
}
```

```

.footer-basic .social>a {
  font-size: 24px;
  width: 40px;
  height: 40px;
  line-height: 40px;
  display: inline-block;
  text-align: center;
  border-radius: 50%;
  border: 1px solid #ccc;
  margin: 0 8px;
  color: inherit;
  opacity: 0.75;
}

.footer-basic .social>a:hover {
  opacity: 0.9;
}

.footer-basic .copyright {
  margin-top: 15px;
  text-align: center;
  font-size: 13px;
  color: #aaa;
  margin-bottom: 0;
}
</style>
</head>

<body>
  <nav class="navbar navbar-expand-lg navbar-dark " style="background-color:#05380b;">
    <!-- <a class="navbar-brand" href="#">Navbar</a>
    <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarNavDropdown" aria-controls="navbarNavDropdown" aria-expanded="false" aria-
label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button> -->
    <div class="collapse navbar-collapse" id="navbarNavDropdown">
      <ul class="navbar-nav">
        <li class="navbar-brand active">
          <a class="nav-link" href="/" style="color: white;">Home</a>
        </li>
        <li class="navbar-brand active">
          <a class="nav-link" href="/index" style="color: white;">AI Engine</a>
        </li>
        <li class="navbar-brand">
          <a class="nav-link" href="/market" style="color: white;">Supplements</a>
        </li>
        <li class="navbar-brand">
          <a class="nav-link" href="/contact" style="color: white;">Contact-Us</a>
        </li>
      </ul>
    </div>
  </nav>
  {% block body %}

```

```

{% endblock body %}
<div class="footer-basic">
  <footer>
    <center>
      <div style="padding: 5;">
        <a target="_blank" href="https://www.facebook.com/share/14zdcWUqQe/"></a>&nbsp;
        <a target="_blank" href="https://www.instagram.com/sadanand_kumarmodi/"></a>&nbsp;
        <a target="_blank" href="https://www.linkedin.com/in/sadanand8789"></a>&nbsp;
        <a target="_blank" href="https://github.com/Sadanand87"></a>&nbsp;
      </div>
    </center>
    <br>
    <ul class="list-inline">
      <li class="list-inline-item"><a href="/">Home</a></li>
      <li class="list-inline-item"><a target="_blank" href="">Services</a></li>
      <li class="list-inline-item"><a target="_blank" href="">About</a></li>
      <li class="list-inline-item laptop"><a href="/index">AI Engine</a></li>
    </ul>
    <p class="copyright">Created by Sadanand Kumar </p>
  </footer>
</div>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/twitter-
bootstrap/4.1.3/js/bootstrap.bundle.min.js"></script>
</body>

</html>

```

3.3.4 home.html

```

{% extends 'base.html' %}
{% block pagetitle %}
Plant Disease Detection
{% endblock pagetitle %}
{% block body %}
<style>
  body{
    overflow-x: hidden;
  }
  @media only screen and (max-width: 830px) {
    .laptop{
      display: none;
    }
  }
  @media (min-width:830px) and (max-width: 1536px){

```

```

.mobile{
  display: none;
}
}
</style>
<div class='mobile p-5'>
  <div class="row mb-5 text-center text-white">
    <div class="col-lg-10 mx-auto">
      <h1 class="display-4" style="padding-top: 2%;font-weight: 400;color:red;"><b>Note📌📄</b></h1>
      <p class="lead" style="font-weight: 500;color: black;">Please🖱️🖱️ Open This Site On Laptop/PC 🖥️ Screen
Only</p>
      <p class="lead" style="font-weight: 500;color: black;">Thank You  </p>
    </div>
  </div>
</div>

<div class="row mb-5 text-center text-white laptop">
  <div class="col-lg-10 mx-auto">
    <h1 class="display-4" style="padding-top: 2%;font-weight: 400;color: rgb(4, 54, 4);"><b>🌿Plant Leaf
Disease
  Detection System🌿 </b></h1>
    <p class="lead" style="font-weight: 500;color: black;">This Plant Leaf Disease
  Detection System Will Help To Detect Disease From Following </p>
  </div>
</div>

<center class="laptop">
  <a href="/index">
    <button class = "btnn-pill btnn-lg btnn-success"><b style="color: black;">Plant Leaf Disease
  Detection System</b></button>
  </a>
</center>
<div class="row p-5 laptop">
  <div class="col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">
      
      <center><p class="lead" style="font-weight: 500;color: black;">Apple</p></center>
    </div>
  </div>
  <div class="col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">
      
      <center><p class="lead" style="font-weight: 500;color: black;">Blueberry</p></center>
    </div>
  </div>
  <div class="col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">
      

```

```

        <center><p class="lead" style="font-weight: 500;color: black;">Cherry</p></center>
    </div>
</div>
<div class="col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

        
        <center><p class="lead" style="font-weight: 500;color: black;">Corn</p></center>
    </div>
</div>
</div>

<div class="row p-5 laptop">
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">
            
            <center><p class="lead" style="font-weight: 500;color: black;">Grape</p></center>
        </div>
    </div>
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

            
            <center><p class="lead" style="font-weight: 500;color: black;">Orange</p></center>
        </div>
    </div>
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

            
            <center><p class="lead" style="font-weight: 500;color: black;">Peach</p></center>
        </div>
    </div>
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

            
            <center><p class="lead" style="font-weight: 500;color: black;">Pepper Bell</p></center>
        </div>
    </div>
</div>

<div class="row p-5 laptop">
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">
            
            <center><p class="lead" style="font-weight: 500;color: black;">Potato</p></center>

```

```

    </div>
</div>
<div class="col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

        
        <center><p class="lead" style="font-weight: 500;color: black;">Raspberry</p></center>
    </div>
</div>

<div class=" col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

        
        <center><p class="lead" style="font-weight: 500;color: black;">Soybean</p></center>
    </div>
</div>
<div class=" col">
    <div class="p-3 bg-white shadow rounded-lg" style="width: 90%;height: 92%;">

        
        <center><p class="lead" style="font-weight: 500;color: black;">Squash</p></center>
    </div>
</div>
</div>

<center class="laptop">
<div class="row p-5">
    <div class=" col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 45%;height: 92%;">
            
            <center><p class="lead" style="font-weight: 500;color: black;">Strawberry</p></center>
        </div>
    </div>
    <div class="col">
        <div class="p-3 bg-white shadow rounded-lg" style="width: 45%;height: 92%;">

            
            <center><p class="lead" style="font-weight: 500;color: black;">Tomato</p></center>
        </div>
    </div>

</div>
</center>

{% endblock body %}

```

3.3.5 index.html

```
<html>
{% extends 'base.html' %}
{% block pagetitle %}
AI Engine
{% endblock pagetitle %}

{% block body %}
<div>
  <div class="container">
    <!-- For demo purpose -->
    <div class="row mb-5 text-center text-white">
      <div class="col-lg-10 mx-auto">
        <h1 class="display-4" style="padding-top: 2%;font-weight: 400;color: rgb(4, 54, 4);"><b>🌿Plant
Leaf Disease Detection System🌿</b></h1>
        <p class="lead" style="font-weight: 500;color: black;">Let Plant Leaf Disease Detection System
Will Help You To Detect Disease</p>
      </div>
    </div>
    <!-- End -->
    <div class="row ">
      <div class="col mx-auto">
        <div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">

          <h5><b>Why is it necessary to detect disease in plant ?</b></h5>
          <p>Plant diseases affect the growth of their respective species. In addition, some research gaps
are
          identified from which to obtain greater transparency for detecting diseases in plants, even
          before their symptoms appear clearly.
          diagnosis is one of the most important aspects of a plant pathologist's training. Without proper
          identification of the disease and the disease-causing agent, disease control measures can be a
          waste of time and money and can lead to further plant losses. Proper disease diagnosis is
          necessary.
          </p>
        </div>
      </div>
    </div>
    <div class="col mx-auto">

      <!-- Default bootstrap file upload-->

      <form action="/submit" method="POST" enctype="multipart/form-data">
        <div class="custom-file overflow-hidden mb-4">
          <input type="file" id="actual-btn" hidden name="image" />
          <label for="actual-btn">Choose File</label>
          <label id="camera-btn">Open Camera</label>

          <br>
          <span id="file-chosen">No file chosen</span>
        </div>

        <!-- Camera feed container (hidden initially) -->
```

```

<div id="camera-container" style="display: none;">
  <video id="camera-feed" width="320" height="240" autoplay></video>
  <button type="button" id="capture-btn">Capture Photo</button>
</div>

<!-- Preview Image (will show the captured photo) -->


<!-- End -->

<h6 >
  Simply upload your plant's leaf image and then see the magic of AI.
</h6>

<!-- Custom bootstrap upload file-->
<center>
  <a class="mx-2"><button type="submit" class="btn btn-primary">Submit</button></a>
</center>
</form>
<!-- End -->

</div>
</div>
<div class="col mx-auto">
  <div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">
    <h5><b>Prevent Plant Disease follow below steps:</b></h5>
    <ol>
      <li>Follow Good Sanitation Practices.</li>
      <li>Fertilize to Keep Your Plants Healthy.</li>
      <li>Inspect Plants for Diseases Before You Bring Them Home.</li>
      <li>Allow the Soil to Warm Before Planting.</li>
      <li>Ensure a Healthy Vegetable Garden By Rotating Crops.</li>
      <li>Provide Good Air Circulation</li>
      <li>Remove Diseased Stems and Foliage</li>
    </ol>
    <a target="_blank" href="https://www.thespruce.com/prevent-plant-diseases-in-your-garden-
2539511"
      class="mx-2"><button type="button" class="btn btn-outline-success">More info</button></a>
  </div>
</div>
</div>
</div>
</div>
<script>
  const actualBtn = document.getElementById('actual-btn');
  const captureBtn = document.getElementById('capture-btn');

  const fileChosen = document.getElementById('file-chosen');

  actualBtn.addEventListener('change', function () {
    fileChosen.textContent = this.files[0].name
  });

```



```
}}
```

```
let capturedFile = null; // Variable to hold the captured file
```

```
document.getElementById('camera-btn').addEventListener('click', function () {  
  // Show the camera container  
  document.getElementById('camera-container').style.display = 'block';  
  
  // Start the camera feed  
  startCamera();  
});
```

```
// Start the camera feed using the getUserMedia API  
function startCamera() {  
  const cameraFeed = document.getElementById('camera-feed');  
  
  // Request access to the user's camera  
  navigator.mediaDevices.getUserMedia({ video: true })  
    .then(function (stream) {  
      // Set the camera feed source to the stream  
      cameraFeed.srcObject = stream;  
    })  
    .catch(function (error) {  
      console.log('Error accessing the camera: ', error);  
    });  
}
```

```
// Capture photo from the camera feed  
document.getElementById('capture-btn').addEventListener('click', function () {  
  const cameraFeed = document.getElementById('camera-feed');  
  const canvas = document.createElement('canvas');  
  const ctx = canvas.getContext('2d');  
  
  // Set canvas dimensions to match the video feed dimensions  
  canvas.width = cameraFeed.videoWidth;  
  canvas.height = cameraFeed.videoHeight;  
  
  // Draw the current frame from the video feed onto the canvas  
  ctx.drawImage(cameraFeed, 0, 0, canvas.width, canvas.height);  
  
  // Convert the canvas image to a data URL  
  const dataUrl = canvas.toDataURL('image/jpeg');  
  
  // Create a new file from the data URL  
  const imageBlob = dataURIToBlob(dataUrl);  
  capturedFile = new File([imageBlob], "camera_image.jpg", { type: 'image/jpeg' });  
  
  // Now you have a file object that you can use locally  
  document.getElementById('file-chosen').textContent = capturedFile.name;  
  
  // Show the captured image preview  
  const preview = document.getElementById('preview');  
  preview.src = dataUrl;
```

```

    preview.style.display = 'block'; // Show the preview
  });

  // Helper function to convert data URL to Blob
  function dataURIToBlob(dataURI) {
    const byteString = atob(dataURI.split(',')[1]);
    const arrayBuffer = new ArrayBuffer(byteString.length);
    const uintArray = new Uint8Array(arrayBuffer);
    for (let i = 0; i < byteString.length; i++) {
      uintArray[i] = byteString.charCodeAt(i);
    }
    return new Blob([uintArray], { type: 'image/jpeg' });
  }

  // Set up the "Choose File" input to trigger when the image is captured
  document.getElementById('actual-btn').addEventListener('click', function () {
    // Use the captured image file as the input file for form submission
    if (capturedFile) {
      const dataTransfer = new DataTransfer(); // Create a DataTransfer object
      dataTransfer.items.add(capturedFile); // Add the captured file to the DataTransfer object
      document.getElementById('actual-btn').files = dataTransfer.files; // Assign the file to the input
    }
  });

  // Start the camera feed
  async function startCamera() {
    const videoElement = document.getElementById('camera-feed');
    try {
      const stream = await navigator.mediaDevices.getUserMedia({ video: true });
      videoElement.srcObject = stream;
    } catch (err) {
      console.error('Error accessing the camera: ', err);
    }
  }

  // Capture the image from the video feed
  document.getElementById('capture-btn').addEventListener('click', function () {
    const video = document.getElementById('camera-feed');
    const canvas = document.getElementById('canvas');
    const context = canvas.getContext('2d');

    // Set canvas size equal to video size
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;

    // Draw the video frame onto the canvas
    context.drawImage(video, 0, 0, canvas.width, canvas.height);

    // Convert the image to a data URL and create a file input
    const dataUrl = canvas.toDataURL('image/png');
    const fileInput = document.getElementById('captured-image');
  });

```

```

const file = dataUrlToFile(dataUrl, 'captured-image.png');

// Create a File object from the base64 image data
const dataTransfer = new DataTransfer();
dataTransfer.items.add(file);
fileInput.files = dataTransfer.files;

// Hide the camera feed and display the chosen file text
document.getElementById('camera-container').style.display = 'none';
document.getElementById('file-chosen').innerText = 'Image Captured';
});

// Convert base64 image data to a file object
function dataUrlToFile(dataUrl, filename) {
  const arr = dataUrl.split(',');
  const mime = arr[0].match(/:(.*?);/)[1];
  const bstr = atob(arr[1]);
  const n = bstr.length;
  const u8arr = new Uint8Array(n);

  while (n--) {
    u8arr[n] = bstr.charCodeAt(n);
  }

  return new File([u8arr], filename, { type: mime });
}

```

</script>

{% endblock body %}

3.3.6 submit.html

```

{% extends 'base.html' %}
{% block pagetitle %}
{{title}}
{% endblock pagetitle %}
{% block body %}

```

<div>

<div class="container">

<!-- For demo purpose -->

<div class="row mb-5 text-center text-white">

<div class="col-lg-10 mx-auto">

<h1 class="display-4" style="padding-top: 2%;font-weight: 400;color: rgb(8, 49, 121);">{{title}}</h1>

</div>

</div>

<center>

<div class=" col">

<div class="p-3 bg-white shadow rounded-lg" style="width: 30%;">


```

</div>
</div>
</center>
<br>
<div class=" row ">
  <div class="col mx-auto">
    <div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">

      <h5><b>
        {% if pred==3 or pred==5 or pred==7 or pred==11 or pred==15 or pred==18 or pred==20 or
pred==23 or
        pred==24 or pred==25 or pred==28 or pred==38 %}
        Tips to Grow Healthy Plants :
        {% else %}
        Brief Descriptpion :
        {% endif %}
      </b></h5>
      <p>{{desc}}
      </p>
    </div>
  </div>
</div>
<div class="row">
  <div class="col mx-auto">
    <div class="p-5 bg-white shadow rounded-lg" style="height:95%">
      <h5><b>
        {% if pred==3 or pred==5 or pred==7 or pred==11 or pred==15 or pred==18 or pred==20 or
pred==23 or
        pred==24 or pred==25 or pred==28 or pred==38 %}
        Benefits :
        {% else %}
        Prevent This Plant Disease By follow below steps :
        {% endif %}
      </b></h5>
      <p>
        {{prevent}}
      </p>
    </div>
  </div>

  {% if pred!=4 %}
  <div class="col mx-auto">
    <div class=" p-5 bg-white shadow rounded-lg" style="height: 95%;">
      <center>
        <h5><b>
          {% if pred==3 or pred==5 or pred==7 or pred==11 or pred==15 or pred==18 or pred==20 or
pred==23 or
          pred==24 or pred==25 or pred==28 or pred==38 %}
          Fertilizer :
          {% else %}
          Supplements :
          {% endif %}
        </b></h5>
        <br>

```

```

<img src={{simage}} width="300" height="350">
<br>
<br>
<h6>{{sname}}</h6>

<a target="_blank" href={{buy_link}}><button type="button" class="btn btn-success"
  style="background-color: #096609;">Buy Product</button></a>
</center>
</div>
</div>
{% endif %}

</div>
</div>
</div>
{% endblock body %}

```

```

(PORT, () => console.log(`Server is up and running on port ${PORT}`));

```

CHAPTER 4: OUTPUTS

4.1 Main Home Page of PLANT LEAF DISEASE DETECTION SYSTEM

The main home page of the **PLANT LEAF DISEASE DETECTION SYSTEM** web application serves as the entry point for users. It features a user-friendly interface with navigation options for administrators and user. The home page provides essential information about the platform, including its purpose and functionalities.

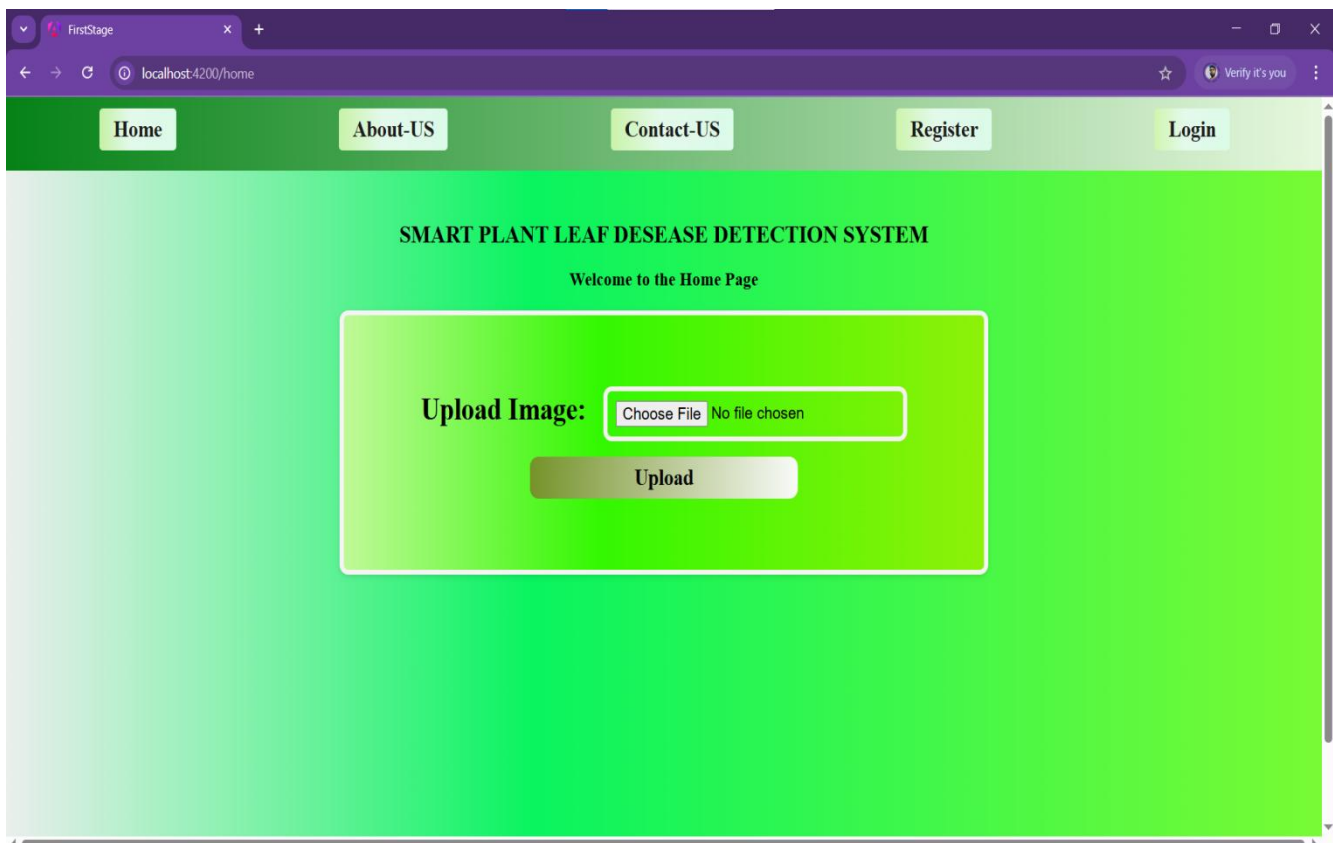


Fig 01: Main Home Page of PLANT LEAF DISEASE DETECTION SYSTEM

4.2 Admin Login Page

The admin login page is designed to ensure high confidentiality and restricted access. It requires a fixed ID and password for authentication, allowing only authorized administrators to manage the system. This security measure prevents unauthorized access to sensitive data and system configurations.

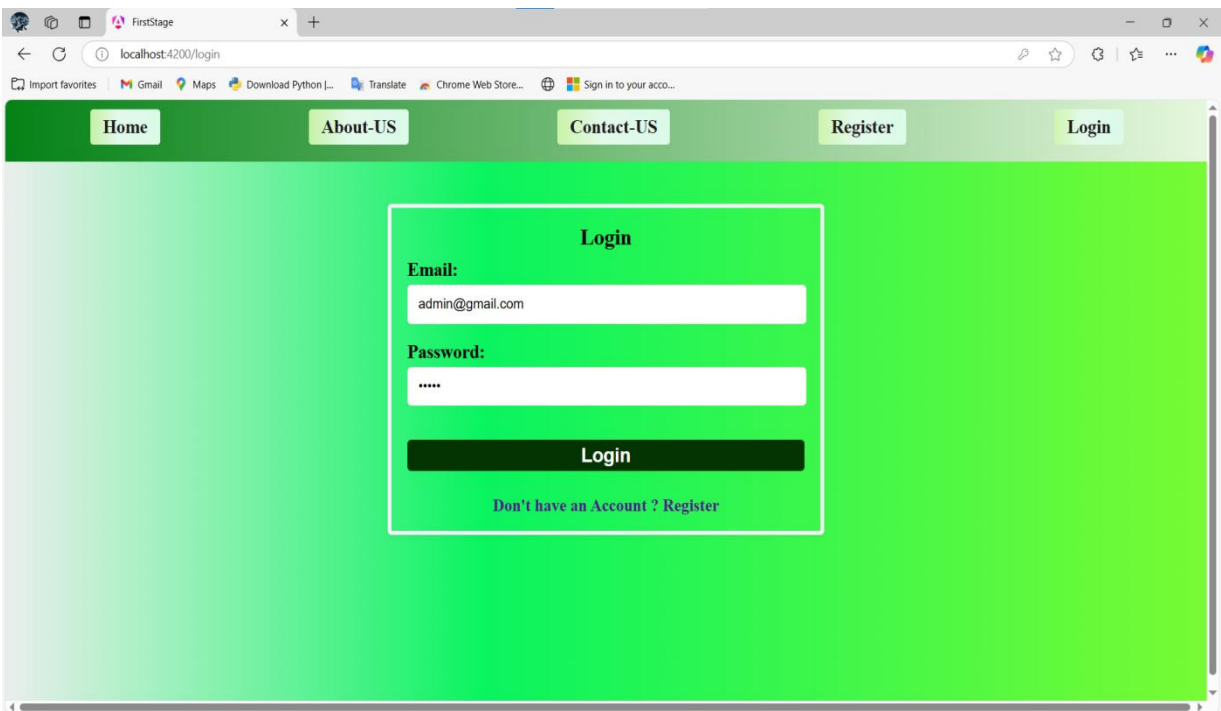


Fig 01:Admin Login

Admin Panel – Subject Creation

In this section, administrators can create subjects and assign passwords to each. Additionally, a unique link is generated for every subject, ensuring secure access for students and faculty. This feature helps in organizing academic sessions efficiently.

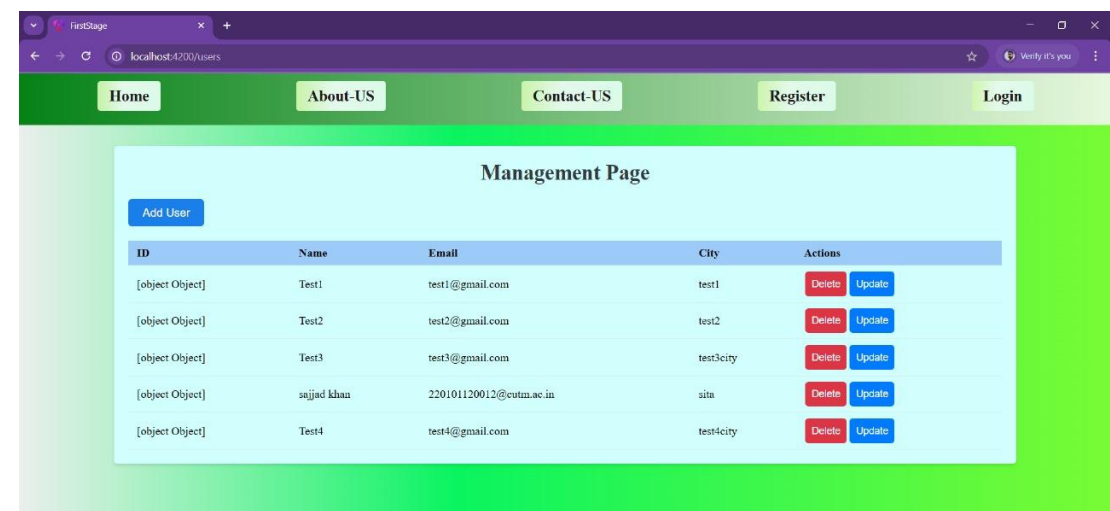


Fig 03: Admin Panel – Subject Creation



Fig 05: Home page

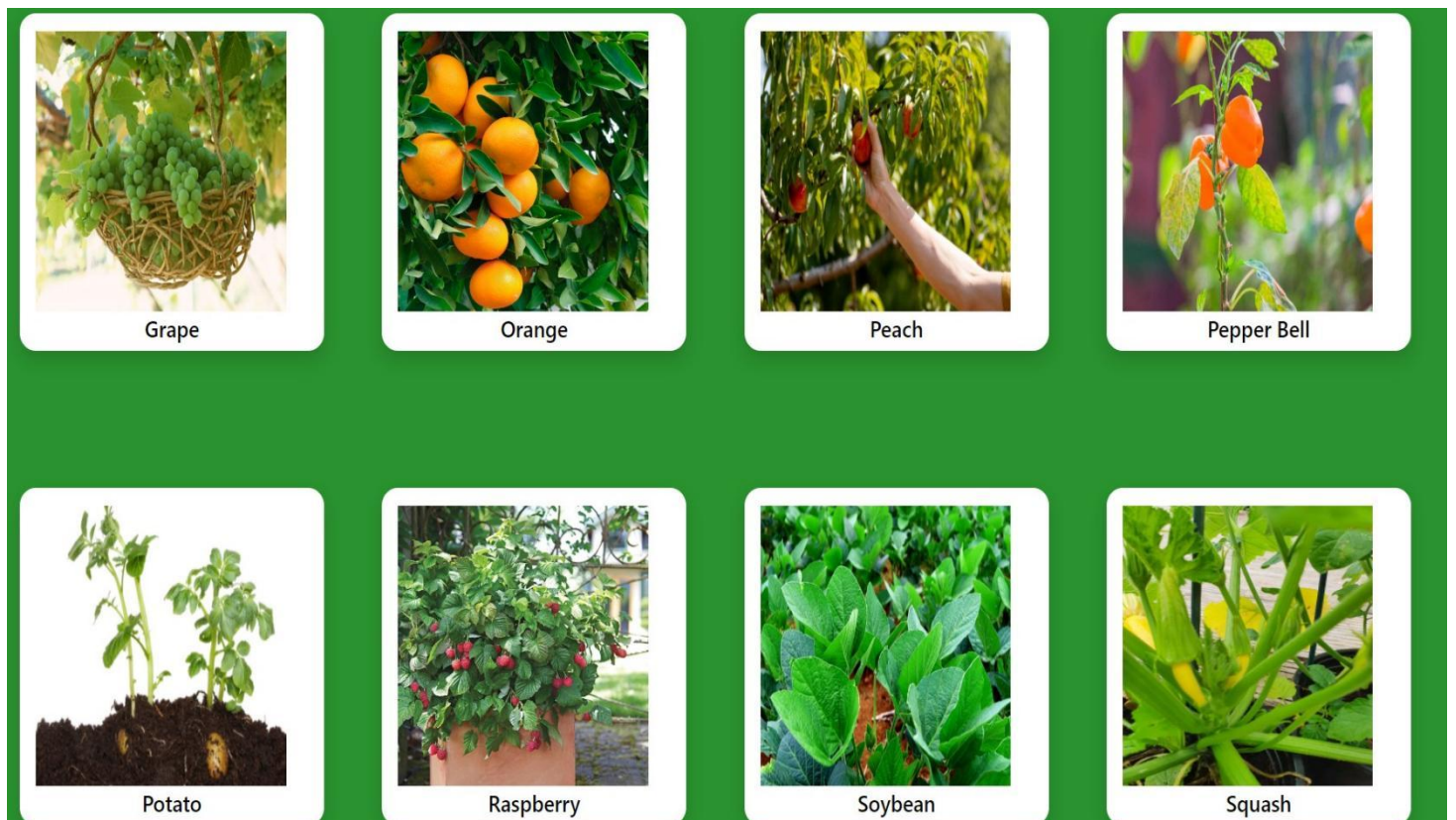


Fig 06: Plant name

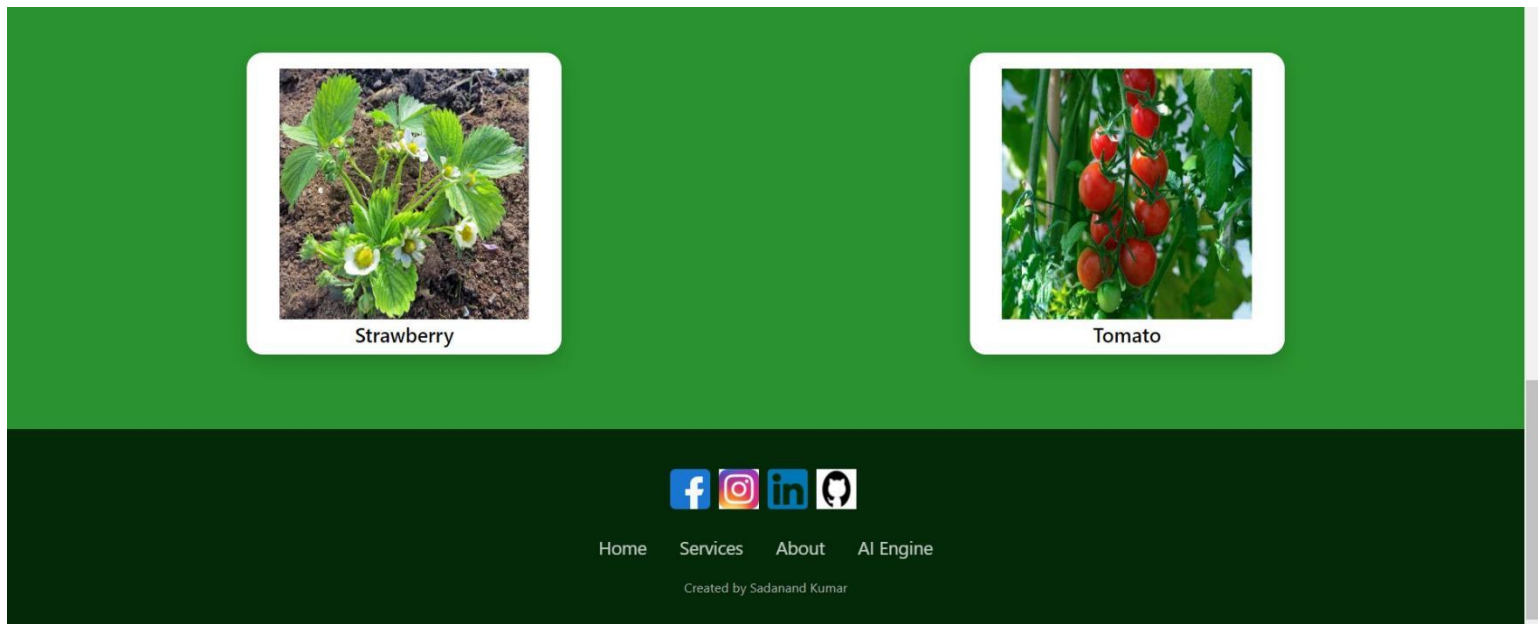


Fig 07: Discription



Fig 08: upload image

Grape : Leaf Blight | Isariopsis Leaf Spot



Brief Description :

The fungus is an obligate pathogen which can attack all green parts of the vine. Symptoms of this disease are frequently confused with those of powdery mildew. Infected leaves develop pale yellow-green lesions which gradually turn brown. Severely infected leaves often drop prematurely. Infected petioles, tendrils, and shoots often curl, develop a shepherd's crook, and eventually turn brown and die. Young berries are highly susceptible to infection and are often covered with white fruiting structures of the fungus. Infected older berries of white cultivars may turn dull gray-green, whereas those of black cultivars turn pinkish red.

Prevent This Plant Disease By follow below steps :

Apply dormant sprays to reduce inoculum levels. Cut it out. Open up that canopy. Don't let down your defenses. Scout early, scout often. Use protectant and systemic fungicides. Consider fungicide resistance. Watch the weather.

Supplements :



Tebulur Tebuconazole 10% + Sulphur 65% WG , Advance Broad Spectrum Premix Fungicides

[Buy Product](#)

Fig 09: Leaf Blight

Supplements

Buy Supplements & Fertilizer at one place

Supplements
(Discontinued)



For Apple - Subi
Karyopore Paste
Fungicide 20% EC
Systemic Fungicide

Buy Product

Supplements
(Discontinued)



For Apple - Black Rot
Magic Fungix For Fungal
Diseases

Buy Product

Supplements
(Discontinued)



For Apple - Cedar Rust
Karyopore All in 1 Organic
Fungicide

Buy Product

Fertilizer (Healthy)



For Apple - Healthy
Leaf Sprayer Organic
Fertilizer

Buy Product

Fertilizer (Healthy)



For Blueberry - Healthy
Bioactive Fertilizer

Buy Product

Supplements
(Discontinued)



For Cherry - Powdery
Mildew
KODAKERIN Fungicide

Buy Product

Fertilizer (Healthy)



For Cherry - Healthy
Plants Organic Bioactive
Liquid Plant Food

Buy Product

Supplements
(Discontinued)



For Citrus - Citrusgreen Leaf
Spot (Grey Leaf Spot)
ANTHRACNOX FUNGICIDE

Buy Product

Supplements
(Discontinued)



For Citrus - Citrus Green Rust
(E. DIAL) (AKA) Mosaiclike 75%
WP Contact Fungicide

Buy Product

Supplements
(Discontinued)



For Citrus - Northern Leaf
Blight
OLUP Contact Fungicide 12% +
Mancozeb 65% WP
Protective And Curative
Fungicide

Buy Product

Fertilizer (Healthy)



For Citrus - Healthy
Borax Leaf Sprayer Fertilizer
Also Multiple Organic
Fertilizer & Plant Food

Buy Product

Supplements
(Discontinued)



For Citrus - Black Rot
Southern Ag Captan 50%
WP Fungicide

Buy Product

Supplements
(Discontinued)



For Citrus - Frost & Black
Mosaic
AURITE FUNGICIDE

Buy Product

Supplements
(Discontinued)



For Citrus - Leaf Blight |
Narrow Leaf Spot
NARROW LEAF SPOT
FUNGICIDE 10% +
Sulfur 65% WP, Active
Broad Spectrum Fungicide
Fungicide

Buy Product

Fertilizer (Healthy)



For Citrus - Healthy
Contact Green Sprayer
Fertilizer Fertilizer

Buy Product

Supplements
(Discontinued)



For Citrus -
Narrowleaf Blight
Green One CITRUS-FAST
FOOD Fertilizer

Buy Product

Supplements
(Discontinued)



For Peach - Bacterial Spot
KODAKERIN FUNGICIDE

Buy Product

Fertilizer (Healthy)



For Peach - Healthy
Insecticide (Plant Growth
Tonic)

Buy Product

Supplements
(Discontinued)



For Pepper Leaf - Bacterial
Spot
Systemic Fungicide (Dinopate)
Tetranebulin 10% WP (10%
WP) WP

Buy Product

Fertilizer (Healthy)



For Pepper Leaf - Healthy
Can Do All in One Organic
Fertilizer Fertilizer

Buy Product

Supplements
(Discontinued)



For Potato - Early Blight
Threats Vertical Fungicide
(With Turmeric Extract)

Buy Product

Supplements
(Discontinued)



For Potato - Late Blight
Sprayable Systemic Fungicide
Fungicide

Buy Product

Fertilizer (Healthy)



For Potato - Healthy
Liquid Fertilizer for potato
Fertilizer

Buy Product

Fertilizer (Healthy)



For Squash - Healthy
Contact Natural Organic
Leaf Fertilizer

Buy Product

Fertilizer (Healthy)



For Squash - Healthy
Max Crop Liquid Fertilizer

Buy Product

Supplements
(Discontinued)



For Squash - Powdery
Mildew
No powdery mildew 1 quart

Buy Product

Supplements
(Discontinued)



For Strawberry - Leaf Scorch
Guaranteed Premium Quality
All in 1 Organic Fertilizer
Fertilizer

Buy Product

Fertilizer (Healthy)



For Strawberry - Healthy
SWEET GREEN ORGANIC
PLANT FOOD (ECONOMY)
FERTILIZER STRAWBERRY
Fertilizer

Buy Product

Supplements
(Discontinued)



For Tomato - Bacterial Spot
CLIFBIO, Bact-Tomato & Bacterium

Buy Product

Supplements
(Discontinued)



For Tomato - Early Blight
NARROW LEAF SPOT

Buy Product

Supplements
(Discontinued)



For Tomato - Late Blight
ACROBAT FUNGICIDE

Buy Product

Supplements
(Discontinued)



For Tomato - Leaf Mold
Virus Sprayer (Leaf of mold) 10% + 10% (10%
+ 10%)

Buy Product

Supplements
(Discontinued)



For Tomato - Septoria Leaf Spot
Leaf Fungicide

Buy Product

Supplements
(Discontinued)



For Tomato - Spider Mites | Two Spotted
Spider Mite
OMATE INSECTICIDE

Buy Product

Supplements
(Discontinued)



For Tomato - Target Spot
Proper Fungicide 75% WP Fungicide for Plants
Disease Control Fertilizer

Buy Product

Supplements
(Discontinued)



For Tomato - Yellow Leaf Curl Virus
Systemic Antiviral Top Fertilizer

Buy Product

Supplements
(Discontinued)



For Tomato - Mosaic Virus
Virus Free (Virus Sprayer)

Buy Product

Fertilizer (Healthy)



For Tomato - Healthy
Tomato-Garden Organic, No Harms, Bacteria,
Tomato & Culture Gardening

Buy Product

Fig 09: Supplement

CHAPTER 5: SUMMARY & SCOPE FOR FURTHER STUDY

5.1SUMMARY

The Plant Leaf Disease Detection System is an AI-powered platform designed to identify and diagnose diseases in plant leaves through advanced image analysis. It provides farmers, gardeners, and agricultural professionals with a user-friendly, efficient solution for early disease detection, enabling timely intervention to protect crops and improve yield. The system leverages machine learning and computer vision technologies to analyze leaf images, detect anomalies, and recommend targeted treatments. It is structured into three core modules—Administrator, Detection, and User—to streamline disease management and user interaction.

Administrator Module

Administrators oversee system operations, manage user accounts, and update the disease database with new information or treatment protocols. They monitor detection accuracy, analyze system performance, and generate reports on disease trends and platform usage. Administrators also ensure data security, manage role-based access, and integrate updates to the machine learning model to enhance diagnostic precision.

Detection Module

This module forms the core of the system, employing trained machine learning models (e.g., CNN, ResNet) to process uploaded leaf images and identify disease patterns. It provides real-time results, highlighting infected areas and classifying diseases with confidence scores. The module also offers actionable insights, such as treatment recommendations, preventive measures, and links to agricultural resources. Historical detection records and image datasets are stored for future reference and model retraining.

User Module

Farmers and gardeners can upload leaf images via web or mobile interfaces, receive instant diagnoses, and access curated solutions. Users view detection histories, track crop health over time, and engage with a knowledge base containing disease descriptions, symptom guides, and agricultural best practices. The platform supports multilingual notifications and alerts for urgent interventions, fostering proactive crop management.

Technology Stack

The system is built using Python for machine learning (TensorFlow, Keras, OpenCV) and the MERN stack (MongoDB, Express.js, React.js, Node.js) for web development. React.js ensures a responsive frontend, while Node.js and Express.js handle backend logic, including API integration for model inference. MongoDB stores user data, detection records, and disease datasets. Cloud storage (e.g., AWS S3) manages image repositories, and secure authentication (JWT, OAuth) ensures data privacy.

Key Features

- **AI-Driven Detection:** Accurate identification of common and rare plant diseases using deep learning.
- **Real-Time Analysis:** Instant results with visual annotations and severity assessment.
- **Treatment Database:** Curated solutions, including organic and chemical treatments.
- **User History:** Track past diagnoses, treatments applied, and outcomes.
- **Multilingual Support:** Accessibility for diverse agricultural communities.

5.2 SCOPE FOR FURTHER STUDY

Plant leaf disease detection systems have shown significant potential in revolutionizing agricultural practices by enabling early diagnosis and mitigation of crop diseases. However, there remains ample scope for advancing research and improving existing methodologies. Below are key areas for further exploration:

1. Enhancement of Dataset Diversity and Quality

Real-world dataset collection: Most systems are trained on lab-condition datasets (e.g., PlantVillage). Expanding to field-captured images with varied lighting, angles, occlusions, and backgrounds would improve robustness.

Multi-species and multi-disease coverage: Developing datasets that include rare diseases, mixed infections, and region-specific crop varieties.

Synthetic data generation: Leveraging Generative Adversarial Networks (GANs) or diffusion models to augment underrepresented classes and simulate challenging field conditions.

2. Advanced Model Architectures and Techniques

Lightweight models for edge deployment: Designing efficient architectures (e.g., MobileNet, EfficientNet) for use on low-power devices in resource-limited farming regions.

Multi-modal learning: Integrating hyperspectral imaging, thermal data, or environmental sensors (e.g., soil moisture, temperature) with visual data for holistic disease analysis.

Few-shot and zero-shot learning: Addressing data scarcity for rare diseases by enabling models to generalize from limited examples.

Explainable AI (XAI): Implementing techniques like Grad-CAM or LIME to interpret model decisions and build trust among farmers.

3. Real-Time and Scalable Solutions

Edge computing integration: Optimizing systems for real-time inference on IoT devices or drones to enable in-field diagnostics.

Cloud-based collaborative systems: Developing federated learning frameworks to aggregate data from multiple farms while preserving privacy.

4. Integration with Agricultural Ecosystems

Precision agriculture tools: Linking disease detection systems with automated treatment mechanisms (e.g., targeted pesticide sprayers, robotic pruners).

Early warning systems: Combining disease predictions with weather data, satellite imagery, and historical trends to forecast outbreaks.

Blockchain for traceability: Creating immutable records of disease history to aid supply chain management and crop certification.

5. Socio-Technical and Ethical Considerations

Farmer-centric design: Developing multilingual, low-literacy interfaces (voice-based or SMS) to improve accessibility for smallholder farmers.

Bias mitigation: Ensuring models perform equitably across diverse geographies, crop types, and farming practices.

Cost-effectiveness analysis: Studying the economic impact of deploying such systems in low-income regions.

6. Cross-Disciplinary Collaborations

Plant pathology integration: Partnering with biologists to validate detected diseases and refine symptom annotations.

Ecological impact studies: Assessing how AI-driven interventions affect biodiversity and long-term soil health.

Policy frameworks: Collaborating with governments to align technology adoption with agricultural subsidies and regulations.

7. Sustainability and Climate Resilience

Climate-adaptive models: Training systems to account for disease patterns altered by climate change.

Carbon footprint reduction: Optimizing training pipelines for energy efficiency and exploring green AI practices

CONCLUSION

The Plant Leaf Disease Detection System has been successfully developed as an innovative, AI-driven solution to address critical challenges in agriculture by enabling early and accurate diagnosis of plant diseases. Leveraging cutting-edge technologies such as Python-based machine learning frameworks (TensorFlow, Keras), computer vision (OpenCV), and the MERN stack for web integration, the platform offers a scalable, secure, and user-friendly tool for farmers, agronomists, and agricultural institutions. Its modular architecture—comprising the Administrator, Detection, and User modules—ensures a streamlined workflow for disease identification, treatment recommendation, and crop health management.

The Administrator Module provides robust oversight, allowing seamless management of user accounts, disease databases, and system security. Administrators ensure the platform's accuracy by updating AI models with new datasets and monitoring detection performance, maintaining alignment with evolving agricultural needs. The Detection Module, powered by deep learning algorithms, delivers real-time, reliable diagnoses of leaf diseases, empowering users with actionable insights such as treatment protocols and preventive measures. This module's ability to process images rapidly and classify diseases with high precision marks a significant advancement in agricultural technology. The User Module prioritizes accessibility, enabling farmers to upload images, receive instant diagnoses, and track crop health histories through an intuitive interface. Features like multilingual support and offline functionality bridge the gap between technology and rural farming communities, ensuring inclusivity.

A key achievement of this system is its integration of secure authentication and role-based access control, safeguarding sensitive user data and detection records. The platform's real-time diagnostic capabilities, combined with cloud-based storage for historical data and scalable image processing, ensure efficiency and reliability even in resource-constrained environments. By providing farmers with immediate, science-backed solutions, the system reduces reliance on guesswork, minimizes crop losses, and promotes sustainable farming practices.

The Plant Leaf Disease Detection System not only addresses immediate agricultural challenges but also lays the groundwork for future innovations in precision farming. Its success demonstrates the transformative potential of AI in bridging the gap between technology and agriculture, offering a scalable model for global food security. As the platform evolves, it will continue to empower farmers, enhance crop resilience, and contribute to a more sustainable and productive agricultural ecosystem. This project underscores the critical role of technology in solving real-world problems, paving the way for smarter, data-driven farming practices worldwide.

FUTURE SCOPE

The Plant Leaf Disease Detection System holds immense potential for expansion and innovation, ensuring it remains a cutting-edge tool for agricultural advancement. As technology progresses, the following enhancements can elevate its impact, accuracy, and accessibility:

1. Advanced AI/ML Integration

Hyperlocal Disease Prediction: Integrate AI with geospatial and environmental data (e.g., soil health, weather patterns) to predict disease outbreaks specific to regions or microclimates.

Multimodal AI Models: Combine image analysis with text (farmers' notes) and sensor data (IoT devices) for holistic disease diagnosis.

Edge AI: Deploy lightweight ML models on mobile devices to enable real-time detection without internet dependency, ideal for remote farming areas.

2. IoT and Drone Integration

Automated Field Monitoring: Use IoT-enabled drones or ground robots to capture high-resolution crop images and sensor data (humidity, temperature) for large-scale, real-time disease surveillance.

Smart Farming Ecosystems: Link the system with IoT-based irrigation or pesticide sprayers to trigger automated responses based on disease severity.

3. Global Crop and Disease Expansion

Multi-Crop Support: Extend the model to cover fruits, vegetables, cash crops, and forestry plants, addressing diverse agricultural needs.

Regional Adaptability: Collaborate with international agricultural agencies to train models on region-specific diseases and pests, ensuring global relevance.

4. Augmented Reality (AR) for Field Guidance

AR Overlay Solutions: Develop mobile AR features to overlay treatment steps or disease information directly onto a farmer's camera view, aiding in-field decision-making.

3D Symptom Visualization: Allow users to interact with 3D models of disease progression for better understanding.

5. Blockchain for Transparency and Trust

Supply Chain Integration: Use blockchain to track disease history and treatment records, enabling buyers to verify crop quality and organic compliance.

Decentralized Data Sharing: Create a secure, global repository for anonymized disease data, fostering collaborative research while protecting farmer privacy.

6. Mobile-First and Offline Capabilities

Low-Bandwidth Optimization: Design a lightweight mobile app with offline image analysis, voice-guided navigation, and multilingual support for rural users.

SMS-Based Alerts: Deliver disease warnings and preventive tips via SMS to farmers without smartphones.

7. Climate Resilience and Sustainability

Eco-Friendly Treatment Suggestions: Prioritize recommendations for organic or low-chemical treatments to promote sustainable farming.

Carbon Footprint Analysis: Integrate tools to measure the environmental impact of treatments, helping farmers adopt eco-conscious practices.

8. Farmer-Expert Collaboration

Teleconsultation Hub: Connect farmers with agronomists via in-app video calls for personalized advice and second opinions on complex cases.

Community Knowledge Base: Let farmers share success stories, home remedies, and localized solutions through a moderated forum.

9. Gamification and Incentives

Farmer Rewards Program: Offer incentives (e.g., discounts on seeds, certifications) for early disease reporting or adopting sustainable practices.

Interactive Learning Modules: Use gamified tutorials to educate farmers on disease prevention and technology usage.

10. Policy and Research Partnerships

Government Collaboration: Partner with agricultural ministries to integrate the system into national farming initiatives, subsidy programs, or disaster response plans.

Academic Research: Enable universities to contribute to model training, validation, and open-source tool development.

REFERENCES

[1] Anand H. Kulkarni, Ashwin Patil R. K., Applying image processing technique to detect plant diseases,

International Journal of Modern Engineering Research, vol.2, Issue.5, pp: 3661-3664, 2012.

[2] Tushar H. Jaware, Ravindra D. Badgujar and Prashant G. Patil, Crop disease detection using image

segmentation, National Conference on Advances in Communication and Computing, World Journal of Science and

Technology, pp:190- 194, Dhule, Maharashtra, India, 2012.

[3] Prof.Sanjay B. Dhaygude, Mr.Nitin P. Kumbhar, Agricultural plant Leaf Disease Detection Using Image

Processing, International Journal of Advanced Research in Electrical, Electronics and Instrumentation

Engineering , S & S Publication vol. 2, Issue 1, pp: 599- 602, 2013.

ASSESSMENT

Internal:

SL NO	RUBRICS	FULL MARK	MARKS OBTAINED	REMARKS
1	Understanding the relevance, scope and dimension of the project	10		
2	Methodology	10		
3	Quality of Analysis and Results	10		
4	Interpretations and Conclusions	10		
5	Report	10		
	Total	50		

Date:

Signature of the Faculty

COURSE OUTCOME (COs) ATTAINMENT

➤ Expected Course Outcomes (COs):

(Refer to COs Statement in the Syllabus)

Understand the core concepts and architecture of Angular applications. Solving the

Develop Angular applications using TypeScript and advanced Angular features .

Integrate Java applications with databases using JDBC and manage transactions.

Secure Spring Boot applications and perform testing.

➤ Course Outcome Attained:

How would you rate your learning of the subject based on the specified COs?

☐

1

☐

2

☐

3

☐

4

☐

5

☐

6

☐

7

☐

8

☐

9

☒

10

LOW

HIGH

➤ Learning Gap (if any):

No Gap

➤ Books / Manuals Referred:

1. "Spring Boot in Action" by Craig Walls.

2 "Angular Up & Running" by Shyam Seshadri

3."Core Servlets and Java Server Pages" by Marty Hall and Larry Brown

Date:

Signature of the Student

➤ Suggestions / Recommendations:

(By the Course Faculty)

Date:

Signature of the Faculty